PHILIPPINE WOMEN'S COLLEGE
Information Technology Department

DAVAO BANK

A Mobile Banking System Rooted in Davao City Culture

Capstone System Documentation

A Capstone Project Presented to the Faculty of
the Information Technology Department, Philippine Women's College
In Partial Fulfillment of the Requirements for the Subject
COMPLEX SYSTEMS
Bachelor of Information Technology

Prepared by
Ron Albert Ancheta
Justin Arcete

Adviser: Iluminado Canoy

November 2026

Version 1.0 · documents the system as of 4 November 2026

Abstract

Davao Bank is a simulated mobile banking system developed as an undergraduate capstone project. It consists of a customer mobile application built with React Native and Expo, a staff administration console built with Next.js, and a shared Supabase backend that provides authentication, a PostgreSQL database, Row Level Security and server-side database functions. The system takes its name and visual identity from Davao City and its Kadayawan Festival.

The project addresses a problem that general-purpose application tutorials rarely confront: a banking application is only useful if its balances are always correct, its transfers can never be half-completed or duplicated, and each customer can see only his or her own data. Davao Bank therefore keeps all money logic on the server. Balances are never stored as an editable number; they are calculated from an append-only ledger. Every money operation runs inside a single database transaction, locks the affected accounts, carries an idempotency key that prevents duplicate submissions, and writes an audit record.

As of 4 November 2026 the system supports registration and email verification, balance viewing, internal transfers, simulated cash-in and cash-out, simulated bill payment, one-time and recurring scheduled payments, QR-based payment and money requests, transaction history, PDF and CSV statements, in-app notifications, and a view-only foreign exchange reference chart. Staff can manage users and accounts, adjust balances with a recorded reason, reverse transfers, monitor schedules and billers, and review the audit log.

Testing combined an offline database harness (63 of 63 checks passing on 4 November 2026), unit tests (50 of 50 passing), scripted tests against the hosted database, and observation of the running application on an iOS simulator. The first live run on 3 November 2026 passed 165 of 174 checks; all nine failures were traced to errors in the test scripts rather than in the system, and the concurrency checks for parallel withdrawals, double reversal and the scheduler all passed. Several user-interface defects and verification gaps remain and are documented in Chapter 5. All money in Davao Bank is play money: the system performs no real settlement and is not a licensed bank.

Keywords: mobile banking, double-entry ledger, Row Level Security, idempotency, React Native, Supabase, Kadayawan

Table of Contents

Chapter 1. Introduction **1**
1.1 Project Background 1
1.2 Statement of the Problem 1
1.3 Objectives of the Project 2
1.3.1 General Objective 2
1.3.2 Specific Objectives 2
1.4 Significance of the Project 2
1.5 Scope and Limitations 2
1.5.1 Scope 2
1.5.2 Limitations 3
1.6 Definition of Terms 3

Chapter 2. Project Requirements and System Analysis **5**
2.1 Intended Users and Stakeholders 5
2.2 The Existing Problem and Process 5
2.3 The Proposed System 5
2.4 Functional Requirements 6
2.5 Nonfunctional Requirements 7
2.6 Feasibility and Constraints 7
2.7 Differences Between the Proposal and the Implementation 7

Chapter 3. Development Methodology and System Design **9**
3.1 Development Methodology 9
3.2 Technology Stack 10
3.3 System Architecture 10
3.4 Use Case Diagrams 11
3.5 Data Flow Diagrams 12
3.6 Database Design and Entity Relationship Diagrams 14
3.7 User Interface Design 16
3.7.1 Design Principles 16
3.7.2 The Kadayawan Visual Identity 16
3.7.3 Navigation Structure 16
3.7.4 Principal Screens 17
3.7.5 Accessibility Considerations and Known Gaps 18

Chapter 4. System Implementation **19**
4.1 Application Structure 19
4.2 Customer Mobile Application 19
4.2.1 Navigation and Screen Organisation 19
4.2.2 State Management 19
4.2.3 Connection to Supabase 19
4.2.4 Confirmation Before Money Moves 20
4.2.5 Error Handling 20
4.3 Detailed Feature Documentation: Mobile Application 20

4.3.1 App Launch and Session Restore 20
4.3.2 Registration (Sign Up) 20
4.3.3 Email Verification 21
4.3.4 Sign In, Password Reset and Sign Out 21
4.3.5 Home Dashboard and Balance 21
4.3.6 Transfer 22
4.3.7 Cash-in (Simulated Deposit) 23
4.3.8 Cash-out (Simulated Withdrawal) 23
4.3.9 Pay Bills 24
4.3.10 Scheduled Payments 24
4.3.11 QR Hub: My QR, Scan and Request Money 25
4.3.12 Payment Requests 25
4.3.13 Activity and Transaction Receipt 26
4.3.14 Statement Export 26
4.3.15 Inbox Notifications 27
4.3.16 Profile, Saved Recipients and Settings 27
4.3.17 FX Reference Rates 28
4.3.18 Suspended Account 28
4.4 Staff Administration System 28
4.4.1 Architecture of the Console 28
4.4.2 Staff Sign-in 30
4.4.3 Dashboard 31
4.4.4 User Management 32
4.4.5 Account Management and Balance Adjustment 33
4.4.6 Transaction Monitoring, Reversal and Export 34
4.4.7 Scheduled Payments, Billers and Audit Log 35
4.5 Backend and Database Operations 36
4.5.1 The Money Engine 36
4.5.2 Ledger Model 37
4.5.3 Limits and Business Rules 38
4.5.4 Scheduled Payment Execution 38
4.5.5 Exchange-Rate Synchronisation 39
4.5.6 Notifications 39
4.5.7 Audit Trail 40
4.5.8 Payment Request Lifecycle 41
4.6 Authentication, Security and Validation 41
4.6.1 Authentication 41
4.6.2 Authorization 41
4.6.3 Validation Layers 41
4.6.4 Secrets and Configuration 41
4.6.5 Security Work in Progress 42
4.7 Integration of System Components 42

Chapter 5. Testing and Evaluation 45
5.1 Testing Approach 45
5.2 Test Environment 45
5.3 Test Cases 45

5.3.1 Authentication and Authorization 45
5.3.2 Transaction Logic and Integrity 46
5.3.3 Input Validation and Business Rules 46
5.3.4 Administrative Functions 47
5.3.5 User Interface Behaviour 47
5.4 Actual Results 48
5.5 System Evaluation 48
5.6 Known Issues and Limitations 48
5.7 Outstanding Verification 49

Chapter 6. Summary, Conclusions and Recommendations 50
6.1 Summary of Accomplishments 50
6.2 Alignment With the Original Objectives 50
6.3 Conclusions 50
6.4 Current Limitations 50
6.5 Recommendations 51
6.5.1 Immediate 51
6.5.2 Next Phases 51
6.5.3 Toward Production (Outside the Capstone Scope) 51

Appendix A. Data Dictionary 52
profiles 52
accounts 52
transactions 52
ledger_entries 53
transfers 53
audit_logs 53
notifications 53
saved_recipients 54
billers 54
bill_payments 54
scheduled_payments 54
scheduled_payment_runs 55
payment_requests 55
fx_rates 56
health_check 56

Appendix B. Database Function Reference 57

Appendix C. Row Level Security Policies 58

Appendix D. Diagram Sources 59

Appendix E. Offline Database Test Inventory 60

Appendix F. Installation and Configuration Guide 62

Appendix G. User Guide (Customer) 63
Getting started 63
Sending money 63

Other tasks 63

Appendix H. Administrator Guide **64**

Appendix I. References **65**

List of Figures

Figure 1. System architecture of Davao Bank 11
Figure 2. Use case diagram: customer 12
Figure 3. Use case diagram: administrator and scheduled jobs 12
Figure 4. Context diagram (DFD level 0) 13
Figure 5. Data flow diagram level 1 (rotated to fit the page) 13
Figure 6. Entity relationship diagram: core banking entities 14
Figure 7. Entity relationship diagram: features added in Phase 5 (rotated to fit the page) 15
Figure 8. Mobile app navigation and screen flow 17
Figure 9. Sign-in screen, Home screen, and Home screen scrolled to recent activity (iOS simulator, 3 Nov 2026) 17
Figure 10. Transfer: recipient, amount and review steps (iOS simulator, 3 Nov 2026) 22
Figure 11. Cash-in review and result screens, and the Cash-out input step 23
Figure 12. Pay Bills biller list, Scheduled Payments list, and the QR hub 24
Figure 13. Activity tab, transaction receipt, and Inbox tab (iOS simulator, 3 Nov 2026) 26
Figure 14. Profile tab and FX reference rates screen 27
Figure 15. Administrative workflow in the staff console 29
Figure 16. Staff console sign-in page 30
Figure 17. Staff console dashboard (local development server, 4 Nov 2026) 31
Figure 18. User and customer directory. Personal email addresses and phone numbers have been masked for this document. 32
Figure 19. Deposit and ledger accounts. Personal email addresses have been masked. 33
Figure 20. Transaction oversight and reversals 34
Figure 21. Scheduled payment oversight (left) and biller directory (right) 35
Figure 22. Audit trail 35
Figure 23. Activity diagram: transfer from the mobile app 36
Figure 24. State diagram: transaction status 37
Figure 25. Activity diagram: scheduled payment executor 39
Figure 26. State diagram: payment request lifecycle 41
Figure 27. Sequence diagram: immediate transfer (rotated to fit the page) 43
Figure 28. Sequence diagram: administrator reverses a transfer (rotated to fit the page) 44

List of Tables

Table 1. Specific objectives and measures 2
Table 2. Prototype compared with a production banking system 3
Table 3. Definition of terms 3
Table 4. Users and stakeholders 5
Table 5. Functional requirements and implementation status 6
Table 6. Nonfunctional requirements 7
Table 7. Proposal compared with the implementation 8
Table 8. Development phases and timeline 9
Table 9. Technology stack 10
Table 10. Key architectural decisions 11
Table 11. Summary of database tables 15
Table 12. Core color tokens 16
Table 13. Admin console routes 29
Table 14. Ledger effect of each operation 37
Table 15. Limits enforced by the database 38
Table 16. Notification events 40
Table 17. Audited actions 40
Table 18. Evidence types used in this chapter 45
Table 19. Test cases: authentication and authorization 45
Table 20. Test cases: transaction logic and integrity 46
Table 21. Test cases: validation 46
Table 22. Test cases: administrative functions 47
Table 23. Test cases: user interface (iOS simulator) 47
Table 24. Summary of recorded test runs 48
Table 25. Evaluation against the proposal's success metrics 48
Table 26. Known issues 49
Table 27. Achievement of the specific objectives 50
Table 28. Data dictionary: profiles 52
Table 29. Data dictionary: accounts 52
Table 30. Data dictionary: transactions 52
Table 31. Data dictionary: ledger_entries 53
Table 32. Data dictionary: transfers 53
Table 33. Data dictionary: audit_logs 53
Table 34. Data dictionary: notifications 53
Table 35. Data dictionary: saved_recipients 54
Table 36. Data dictionary: billers 54
Table 37. Data dictionary: bill_payments 54
Table 38. Data dictionary: scheduled_payments 55
Table 39. Data dictionary: scheduled_payment_runs 55
Table 40. Data dictionary: payment_requests 55
Table 41. Data dictionary: fx_rates 56
Table 42. Data dictionary: health_check 56
Table 43. Customer-callable functions 57
Table 44. Administrator functions 57

Table 45. Internal and trigger functions (not callable by clients) 57
Table 46. Row Level Security policies by table 58
Table 47. Diagram source files 59
Table 48. Offline harness checks 60

Chapter 1. Introduction

1.1 Project Background

Mobile banking is now the main way many Filipinos check balances, send money and pay bills. Behind every one of those simple screens is a set of guarantees that users take for granted: the balance shown is correct, money sent is either fully delivered or not sent at all, pressing a button twice does not send the money twice, and no other customer can read one's account. When any of these guarantees fails, the application has failed at its core purpose, regardless of how attractive its screens are.

Davao Bank was conceived as a capstone project that would take those guarantees seriously while remaining an academic prototype. It is a complete, demonstrable banking system with two front ends and one shared backend: a mobile application for customers and a web console for bank staff, both connected to a Supabase project that hosts the database, authentication and server-side logic.

The project also carries a cultural intent. Its name and appearance draw on Davao City and the Kadayawan Festival, the city's annual thanksgiving for a bountiful harvest. The 2026 festival theme, "We Are One, KadayawOne," celebrates the unity of Davao's eleven ethnolinguistic tribes, settlers and guests. The project adopts this idea of many threads woven into one fabric as its guiding metaphor: many individual customers brought together in one trusted system. The interface uses the Kadayawan 2026 color palette, with Mt. Apo green as the primary action color.

From the beginning the team treated Davao Bank as a simulation. Balances represent play money, no real funds are settled, and the system makes no claim to be a licensed financial institution. Within that boundary, the project applies the engineering practices a real bank would require: secure authentication, role-based access, atomic and idempotent money movement, an auditable ledger and layered testing.

1.2 Statement of the Problem

Any team building a banking application, even a prototype, faces a recurring set of problems that ordinary applications can usually ignore. The project proposal identified six, and they remained the problems the implementation was designed around:

1. **Data integrity.** A customer's balance must always equal the sum of his or her transactions. If balances are stored as a field the application edits directly, a single missed update, race condition or crash leaves the data permanently wrong.
2. **Transaction reliability.** Money must move atomically. The sender is debited and the receiver credited together, or neither happens.
3. **Duplicate operations.** A user who taps "Send" twice, or a network retry, must not create two transfers.
4. **Unauthorized access.** Each customer must see and act on only his or her own accounts and transactions. A weak access rule can expose every customer's data at once.
5. **Auditability.** Every money movement and administrative action must be recorded in a way that clients cannot alter, so that history can be reviewed and reconciled.
6. **Confusing a prototype with a production bank.** A managed backend such as Supabase provides authentication, a database and hosting. It does not provide a banking licence, payment rails, identity verification (KYC/AML) or payment-card compliance. Treating the two as equivalent leads to unsafe assumptions.

Stated as research questions, the project sought to answer the following: How can a mobile banking prototype guarantee correct balances, atomic transfers and duplicate-free submissions when clients may retry, crash or act concurrently? How can access be restricted so that customers reach only their own records while staff can administer the whole system? And how can these guarantees be demonstrated and verified, rather than merely asserted?

1.3 Objectives of the Project

1.3.1 General Objective

To design, develop and evaluate Davao Bank, a simulated mobile banking system rooted in Davao City culture, consisting of a customer mobile application, a staff administration console and a shared Supabase backend, in which balances are correct by construction, every money movement is atomic, idempotent and auditable, and every user can access only the data his or her role permits.

1.3.2 Specific Objectives

The proposal defined seven specific objectives, each with a measure. Chapter 6 reports how far each was achieved.

Table 1. Specific objectives and measures

ID	Objective	Measure (from the proposal)
O1	Provide secure authentication and session management for customers and staff	100% of data access requires a valid session; email verification required; no unauthenticated data endpoints
O2	Enforce role-based access so users reach only permitted data	Customers access only their own records; all cross-user and unauthorized-write attempts blocked in tests
O3	Guarantee atomic, correct money movement	0 balance inconsistencies in reconciliation tests; every transfer is all-or-nothing
O4	Prevent duplicate transactions	A repeated submission produces exactly one transaction
O5	Record a complete, tamper-evident audit trail	100% of money movements and admin actions recorded in an append-only log
O6	Deliver a reliable, responsive user experience	Key screens load in under 2 seconds on the test device; all flows handle errors gracefully
O7	Deliver a complete, demonstrable prototype	100% of in-scope features working end to end across mobile, admin and backend

1.4 Significance of the Project

For the developers, the project is an exercise in building software where correctness matters more than features. It required designing a database before screens, writing server-side functions that hold up under concurrent use, and verifying claims with tests against a real database.

For the capstone adviser and panel, the system and this document provide a concrete case study of how integrity, access control and auditability can be enforced in a modern serverless architecture, and of where a prototype stops short of a production bank.

For future developers, the documentation describes the actual implementation in enough detail to maintain or extend it. The database design, money engine and security model are reusable patterns for any application that moves value between users.

For the Davao community, the project shows how local culture can shape the identity of a digital product. The Kadayawan palette, the Bisaya greeting "Maayong adlaw" on the Home screen, and the planned characters Kada (a Philippine eagle) and Duri (a durian) give the application a recognisably Davaoeño character.

1.5 Scope and Limitations

1.5.1 Scope

The delivered system covers three components. The **customer mobile application** provides registration, email verification, sign-in and password-reset requests; a Home dashboard with balances; internal transfers by account number, saved recipient or own account; simulated cash-in and cash-out; simulated bill payment to demonstration billers; one-time and recurring scheduled payments; a QR hub for showing and scanning payment codes and requesting money; transaction history with receipts; PDF and CSV statements; in-app notifications; profile

management; biometric or password confirmation before money moves; and a view-only foreign-exchange reference chart.

The **staff administration console** provides staff-only sign-in, a dashboard of key figures, user management (suspend and reactivate, open accounts), account management (freeze, unfreeze, close, and adjust balances with a mandatory reason), transaction monitoring and reversal, oversight of scheduled payments and payment requests, biller management, a CSV export of transactions, and an audit-log viewer.

The **Supabase backend** provides email and password authentication; a PostgreSQL database of 15 tables with Row Level Security on every table; 39 database functions, including a single money engine; scheduled jobs (pg_cron) that run due payments, expire old payment requests and refresh exchange rates; and one Edge Function that fetches reference rates from the European Central Bank through the Frankfurter API.

1.5.2 Limitations

The following limitations are deliberate and follow from the project's status as an academic prototype:

- All money is simulated. Cash-in and cash-out create and remove play money; no bank, e-wallet, card network, InstaPay, PESONet or other payment rail is connected.
- Bill payments go only to fictional demonstration billers (for example "Mindanao Power Co. (demo)"). No real utility is paid.
- The system operates in a single currency, Philippine pesos. Foreign exchange rates are shown for reference only; the system cannot convert or hold other currencies.
- Identity is limited to email and password with email verification. There is no know-your-customer (KYC) or anti-money-laundering (AML) screening.
- The system has not been deployed to production hosting or published to an app store. During development the mobile app runs in Expo Go and the admin console runs on a local development server, both connected to a hosted Supabase project.
- The prototype has been exercised mainly on an iOS simulator. Android devices, physical phones and accessibility settings such as large text and screen readers have not yet been fully tested (see Section 5.7).

Table 2 summarizes the difference between this prototype and a production banking system. It is carried over from the proposal because it still describes the boundary accurately.

Table 2. Prototype compared with a production banking system

Dimension	Davao Bank (prototype)	Production banking system
Money	Simulated play-money balances; no settlement	Real funds settled through regulated payment rails
Regulation	None; academic use only	Banking licence, consumer-protection and reporting obligations
Identity	Email and password; email verification	Full KYC/AML verification and screening
Payment card data	None stored	PCI-DSS scope, tokenization, strict controls
Core ledger	Ledger in PostgreSQL; balances derived from entries	Dedicated, reconciled core-banking ledger with external audit
Availability	Best effort on free or low-cost managed tiers	High availability, disaster recovery, 24/7 operations
Fraud and risk	Input validation, limits, admin review	Real-time fraud detection, sanctions screening

1.6 Definition of Terms

The following terms are used throughout this document in the sense given here.

Table 3. Definition of terms

Term	Meaning in this document
Account number	A 10-digit identifier beginning with "10", generated by the database for every account.

Term	Meaning in this document
Audit log	The `audit_logs` table. An append-only record of sensitive actions written only by server-side functions.
Centavo	One hundredth of a peso. All amounts are stored as whole centavos to avoid rounding errors (₱1.00 = 100).
Edge Function	A small server-side program hosted by Supabase. Davao Bank has one, `fx-sync`, which fetches exchange rates.
Expo Go	A mobile app that runs Expo projects during development without building a separate installable app.
Idempotency key	A unique string sent with each money request. If the same key arrives twice, the server returns the first result instead of moving money again.
JWT	JSON Web Token. The signed session token Supabase Auth issues after sign-in; the database reads the user's identity from it.
Ledger entry	One line in `ledger_entries`: a positive amount, a direction (debit or credit) and an account. Balances are sums of these lines.
pg_cron	A PostgreSQL extension that runs SQL on a timetable, used for scheduled payments, request expiry and exchange-rate refresh.
Row Level Security (RLS)	A PostgreSQL feature that filters every query by rules (policies) attached to a table, so each user sees only permitted rows.
RPC	Remote procedure call. In Supabase, a database function the app calls over HTTPS, for example `rpc("transfer", ...)`.
SECURITY DEFINER	A PostgreSQL function that runs with its owner's privileges. Money functions use it so that they, and only they, can write to protected tables.
Simulated	Behaving like a banking operation inside the system, without any real money or external institution involved.

Chapter 2. Project Requirements and System Analysis

2.1 Intended Users and Stakeholders

Davao Bank has two kinds of human users and one automated actor. Table 4 lists them together with the project's other stakeholders.

Table 4. Users and stakeholders

User or stakeholder	Where they interact	Role and permissions
Customer	Mobile app	Registers, signs in and manages only his or her own profile, accounts, transactions, schedules, saved recipients, requests and notifications.
Administrator (staff)	Admin web console	Views all users, accounts and transactions; suspends users; opens, freezes and closes accounts; adjusts balances; reverses transactions; manages billers and schedules; reads the audit log. Must have `role = admin` and `status = active`.
System (automated)	Database jobs	pg_cron runs due scheduled payments every 5 minutes, expires payment requests daily and requests fresh exchange rates daily. A `system` profile owns the settlement accounts of the demonstration billers and cannot sign in.
Project owner	Development process	Makes product decisions and approvals and runs the implementing coding agent.
Project manager and technical lead	Development process	Writes phase specifications, reviews each phase against the code and database, and issues fixes.
Capstone adviser and panel	Evaluation	Provide academic oversight and final acceptance.

An Auditor role with read-only access to transactions and the audit log was described in the proposal as a future addition. It was not built.

2.2 The Existing Problem and Process

Davao Bank does not replace an existing institutional system; it is a new prototype. The "existing process" it responds to is the common way small or student banking applications are built: the application reads a balance, subtracts an amount on the phone or in a web page, and writes the new number back. That approach is simple but fails in exactly the situations described in Section 1.2. Two requests that arrive at the same moment both read the old balance and both succeed. A crash between debiting one account and crediting another leaves money missing. A slow network causes the user to tap again and the transfer happens twice. Access rules placed only in the user interface can be bypassed by calling the database directly.

The analysis therefore concluded that the client applications must never be trusted with money. They may display data and submit requests, but the database itself must validate and perform every money movement and enforce who may see what.

2.3 The Proposed System

The system is built around one principle stated in the proposal: **the server is the only authority on money.** Clients never compute or write balances. Every money movement is validated and executed by a database function inside one transaction, and every balance is derived from ledger entries. Access control is enforced in the database with Row Level Security, so it applies equally to the mobile app, the admin console and any other client. Administrative actions also run as database functions that check the caller's role and write an audit record.

The two client applications share a TypeScript package, `@davao-bank/shared`, which contains the Supabase client wrappers, data types, design tokens and utilities for currency, QR codes and idempotency keys. This keeps the rules for formatting money and building requests identical on both clients.

2.4 Functional Requirements

Table 5 lists the functional requirements, combining those in the proposal with those added by the approved scope change of 23 October 2026 (Scope Change 01). The status column reflects the repository on 4 November 2026. "Implemented" means the function exists end to end; "Partial" means part of the requirement is missing; known defects in implemented functions are listed in Section 5.6.

Table 5. Functional requirements and implementation status

ID	Requirement	Source	Status
FR-01	Customers can register with email and password; a profile and a savings account are created automatically	Proposal	Implemented
FR-02	Email verification is required before the app can be used	Proposal	Implemented
FR-03	Customers can request a password reset	Proposal	Partial: reset email is sent, but the reset cannot be completed inside the app
FR-04	Customers see their accounts and a ledger-derived balance, and can hide the balance	Proposal; Scope Change 01	Implemented
FR-05	Customers can transfer to another account by account number with a confirmation step and idempotent submission	Proposal	Implemented
FR-06	Customers can save recipients and transfer between their own accounts	Scope Change 01	Implemented
FR-07	Simulated deposit (cash-in) and withdrawal (cash-out)	Proposal	Implemented
FR-08	Biometric or password confirmation before money moves	Scope Change 01	Implemented
FR-09	Transaction history with filter and detail view	Proposal	Implemented (filter by type; no text search)
FR-10	Simulated bill payment with reference validation	Scope Change 01	Implemented
FR-11	One-time and recurring scheduled transfers and bill payments, executed automatically	Scope Change 01	Implemented
FR-12	Show and scan a Davao Bank QR code to pay; request money from another customer	Scope Change 01	Implemented
FR-13	Account statement export as PDF and CSV	Proposal (optional)	Implemented
FR-14	In-app notifications for money events and account changes	Proposal	Implemented
FR-15	Profile view and edit	Proposal	Implemented (name and phone)
FR-16	View-only foreign exchange reference chart	Scope Change 01	Implemented
FR-17	Staff-only sign-in with role-based access	Proposal	Implemented
FR-18	Dashboard with users, accounts, balances and transaction volume	Proposal	Implemented (two figures have known defects)
FR-19	User management: list, search, view, activate and suspend	Proposal	Implemented
FR-20	Lightweight identity-verification flag for users	Proposal	Not implemented
FR-21	Account management: open, freeze, unfreeze and close	Proposal	Implemented
FR-22	Transaction monitoring and reversal within defined rules	Proposal	Implemented (reversal exposed for transfers only)
FR-23	Flag suspicious transactions	Proposal	Not implemented
FR-24	Manual balance adjustment with a mandatory reason, audit-logged	Proposal	Implemented
FR-25	Audit log viewer and basic report export	Proposal	Partial: viewer implemented; export covers transactions only
FR-26	Staff management of billers and scheduled payments	Scope Change 01	Partial: create biller and schedule tools work; biller activate/deactivate has a defect

2.5 Nonfunctional Requirements

Table 6. Nonfunctional requirements

ID	Requirement	How the design addresses it	Status
NFR-01 Integrity	Balances always equal the ledger	No balance column; balances are sums of ledger entries; writes only through database functions	Met in tests
NFR-02 Atomicity	A transfer completes fully or not at all	Each money function runs in one database transaction	Met in tests
NFR-03 Idempotency	A repeated request produces one transaction	Unique `idempotency_key`; replay returns the original result	Met in tests
NFR-04 Concurrency	Parallel requests cannot overdraw or double-reverse	Row locks in a fixed order; unique index on reversals; run-once keys for schedules	Met in live tests (3 Nov 2026)
NFR-05 Access control	Each role reaches only permitted data	RLS on all 15 tables; `is_admin()` checks; function execute grants	Met in tests
NFR-06 Auditability	Money and admin actions recorded	`write_audit` called inside each function; no client write access	Met, with gaps listed in Section 4.5.7
NFR-07 Security of secrets	No privileged key in client bundles	Clients ship only the public anon key; service key used by scripts and the Edge Function	Met; gitleaks scan in CI
NFR-08 Performance	Key screens load in under 2 seconds	Small queries, indexed foreign keys	Not yet measured
NFR-09 Usability and accessibility	Clear flows, readable text, screen-reader support	Shared design system, confirmation steps, mapped error messages	Partial; see Section 3.7.5
NFR-10 Maintainability	One language and shared code	TypeScript throughout; shared package; append-only migrations; CI checks	Met

2.6 Feasibility and Constraints

Technical feasibility. Every component uses mature, documented technology. Supabase provides a managed PostgreSQL database with Row Level Security and authentication, which removed the need to build and host a custom API server. Expo allowed the mobile application to be developed and demonstrated on simulators and phones through Expo Go without native build tooling.

Economic feasibility. The proposal estimated typical running costs of USD 0 to 50 per month, relying on free tiers. The project has run on a Supabase project and free development tools; no paid services were required during development. Optional costs such as app-store accounts and a custom domain were not incurred.

Operational feasibility. The system is intended for demonstration and evaluation. Customers need only a phone with Expo Go; staff need a web browser. Database jobs run automatically once pg_cron and pg_net are enabled and the job registration function has been executed (Appendix F).

Schedule. The proposal planned 10 weeks in seven phases. The approved scope change added eight features and revised the estimate to about 14 to 15 weeks. Development began on 20 October 2026; Phases 1 to 5 were completed by 3 November 2026, and Phase 6 (testing and hardening) was in progress when this document was written.

Constraints. The project depends on the availability of Supabase and of the Frankfurter API for exchange rates, on the implementing AI coding agent, and on review capacity at each phase gate. Real-money functions are excluded by design.

2.7 Differences Between the Proposal and the Implementation

The proposal (version 1.1, 20 October 2026) describes what the team intended to build. The repository establishes what was actually built. Table 7 lists every significant difference so that no proposed feature is mistaken for completed functionality.

Table 7. Proposal compared with the implementation

Area	Proposal v1.1	Implementation (4 Nov 2026)
Visual identity	Waling-waling purple primary color, with Mt. Apo green, gold, cacao brown and eagle slate	Replaced on 23 Oct 2026 by the Kadayawan 2026 palette with Mt. Apo green (#0C6D38) as primary; Plus Jakarta Sans typeface. The official logo files have not been supplied, so a "LOGO" placeholder is shown
Bill payment	Out of scope	Added by Scope Change 01 as simulated payments to demonstration billers
Foreign exchange	Out of scope (single currency)	Added as a view-only reference chart; the system remains single-currency PHP
Additional features	Not planned	Saved recipients, statements, hide balance, biometric confirmation, scheduled payments, QR send and request money (Scope Change 01)
Biometrics	Optional biometric unlock of the app	Biometric (or password) confirmation before each money movement instead of app unlock
Storage	Supabase Storage for avatars and exported statements	Not used. Avatars show initials; statements are generated on the phone and shared
Ledger	Every transaction's entries net to zero	Transfers, bill payments and request payments are balanced. Deposits, withdrawals and adjustments write a single entry because there is no cash or suspense account (Section 4.5.2)
Roles	Customer and administrator; auditor in future	Customer, administrator and a non-login `system` role for biller settlement accounts; auditor not built
Admin features	Includes a verification flag and flagging suspicious transactions	Neither implemented; audit export not implemented (transaction CSV only)
Transaction search	Search and filter	Filter by type on mobile; text search in the admin console only
Testing tools	Jest, RN Testing Library, pgTAP, Detox and Playwright	Jest and Testing Library; an offline PGlite database harness instead of pgTAP; scripted live suites; Maestro flows instead of Detox; Playwright
Deployment	Admin on Vercel, mobile build with EAS	Not yet done (planned final phase)
Phases	7 phases over 10 weeks	Expanded after the scope change; Phase 6 in progress; UI refinement, workflow, savings and deployment phases planned

Chapter 3. Development Methodology and System Design

3.1 Development Methodology

Davao Bank was developed with an iterative, phase-gated methodology. The work was divided into phases with clear objectives and completion criteria, and no phase could begin until the previous one had been reviewed and approved. This resembles an incremental model with formal review gates rather than a single waterfall pass or open-ended agile sprints. It was chosen because correctness of money handling had to be proven before features were built on top of it.

Implementation was carried out by an AI coding agent working from a detailed written specification (a "phase prompt") for each phase. Each phase followed the same governed cycle:

1. **Specify.** The technical lead wrote the phase prompt: scope, required behaviour, verification steps and an explicit list of things not to change.
2. **Implement.** The coding agent built the phase on its own branch, with one commit and one new database migration per stage.
3. **Review.** The reviewer inspected the actual code, migrations and test output, and from Phase 5 onward also applied the migrations to an independent database harness and ran the app. Reports were never accepted on the agent's summary alone.
4. **Decide.** The phase was marked Approved, Needs Fixes or Requires Further Verification. Fixes were issued as a follow-up prompt.
5. **Proceed.** Only an approved phase unlocked the next one. New requirements were checked against earlier decisions before being accepted.

The review step proved necessary. In Phase 5 the agent's reports claimed passing tests that had in fact run against an empty database in a fallback "contract mode" that only searched source files. The reviewer then built an offline database harness (Section 5.1) and required every later test run to show that it had connected to the live database. Three dedicated fix passes (after Phases 1, 4 and 5) and two carry-over steps resulted from reviews.

Table 8. Development phases and timeline

Phase	Objective and main deliverables	Completed
1 Project setup	Monorepo with mobile, admin and shared packages; Supabase connectivity check; coding standards. Fix pass for workspace and tooling issues	23 Oct 2026 (committed with Phases 2 to 4)
2 Database and authentication	Full schema, RLS on every table, `is_admin()`, sign-up trigger, audit infrastructure, ledger-derived balance view, sign-up and sign-in flows	23 Oct 2026
3 Core banking	Server-authoritative transfer, deposit and withdraw functions with centavo amounts, idempotency keys and row locking; transaction history	23 Oct 2026
4 Admin management	Staff console: dashboard, users, accounts, reversal, adjustments, audit viewer, CSV export. Fix pass for a double-reversal race, unenforced suspension, a forgeable audit function and form idempotency	23 Oct 2026
5 Redesign and feature expansion	Stage A Kadayawan design system; B saved recipients, statements, hide balance, biometric confirmation; C scheduled payments and bill pay on one money engine; D QR and payment requests; E FX reference chart	23 to 27 Oct 2026
5 Fixes	Removed false-pass test mode; fixed seeding, cron registration, request linking, confirmation on devices without biometrics, and recipient tampering	3 Nov 2026
6 Testing and hardening	Stage 0 carry-overs and critical flow fixes; Stage A unit tests, end-to-end flows and CI; Stage B security hardening; Stage C device checks	In progress (Stage 0 and A committed 4 Nov 2026)

On 23 October 2026 the project owner approved Scope Change 01, which added the eight features listed in Table 7 after comparing the design with a major Philippine bank's online banking features. The same record rejected adopting that bank's visual style or a five-tab layout, excluded all real payment networks and real currency conversion, and recorded the expected effect on the timeline.

3.2 Technology Stack

Table 9 lists the technologies actually present in the repository, with the versions declared in its package files. Versions prefixed with a caret (^) or tilde (~) allow compatible minor updates.

Table 9. Technology stack

Layer	Technology and version	Purpose
Language	TypeScript ~5.8.2	Typed code across both clients and the shared package
Mobile framework	React Native 0.88.0, React 19.3.1, Expo SDK ~58.0.16	Cross-platform customer app, run in Expo Go during development
Mobile navigation	@react-navigation/native ^7.0.14, bottom-tabs ^7.21.0	Four-tab navigation
Mobile device APIs	expo-local-authentication, expo-camera, expo-print, expo-sharing, expo-file-system, expo-haptics	Biometrics, QR scanning, PDF generation, sharing, files, haptic feedback
Mobile UI	react-native-svg, react-native-qrcode-svg, lucide-react-native, Plus Jakarta Sans	Charts, QR codes, icons, typeface
Mobile storage	@react-native-async-storage/async-storage 2.2.0	Session and preference storage (encrypted session storage is being added in Phase 6 Stage B)
Admin framework	Next.js 16.4.2 (App Router), React 19.3.1	Server-rendered staff console with server actions
Admin data access	@supabase/ssr ^0.5.2	Cookie-based Supabase sessions in Next.js
Admin UI	CSS Modules, lucide-react ^1.61.0	Styling and icons
Backend	Supabase: Auth, PostgreSQL, PostgREST, Edge Functions (Deno), Vault	Authentication, database, API and server logic
Database extensions	pg_cron, pg_net	Scheduled jobs and outbound HTTP from the database
Client library	@supabase/supabase-js ^2.49.1	Database and auth calls from both clients
External data	Frankfurter API (European Central Bank reference rates)	Exchange-rate history for the FX chart
Testing	Jest ^30.5.2, Testing Library, @electric-sql/pglite ^0.5.12, tsx scripts, Playwright ^1.64.1, Maestro	Unit, offline database, live database and end-to-end tests
Quality tools	ESLint 9, Prettier 3, GitHub Actions, gitleaks	Linting, formatting, continuous integration, secret scanning
Runtime	Node.js >= 20 (24.12.0 used)	Tooling and scripts

These technologies suit the project for three reasons. First, one language, TypeScript, runs on both clients and in the shared package, so money formatting and request rules are written once. Second, Supabase places the business rules next to the data in PostgreSQL, where transactions, row locks and Row Level Security are native features rather than application code that could be bypassed. Third, Expo and Next.js allowed a two-person team to deliver both a mobile app and a web console within the timeline.

3.3 System Architecture

Figure 1 shows the overall architecture. Both clients talk directly to Supabase over HTTPS; there is no separate application server. Reads go through PostgREST, which applies Row Level Security to every query. All writes to money tables go through database functions that run as SECURITY DEFINER, so clients have no direct write access to those tables.

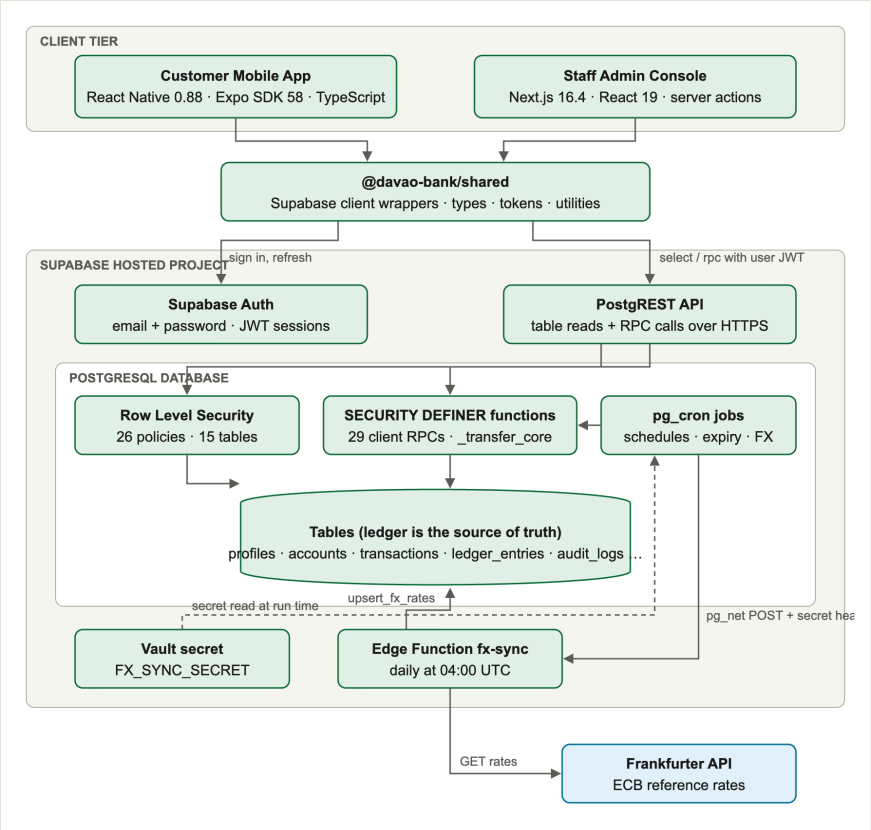


Figure 1. System architecture of Davao Bank

The architecture has four layers. The **client tier** consists of the mobile app and the admin console. The **shared package** gives both clients the same Supabase client wrappers, types and utilities. The **API layer** is provided by Supabase Auth, which issues JSON Web Tokens, and PostgREST, which exposes tables and functions over HTTPS. The **database layer** holds the 15 tables, the Row Level Security policies, the 39 functions and the scheduled jobs. A small **FX synchronisation** path lets a scheduled job call the `fx-sync` Edge Function, which fetches reference rates from the Frankfurter API and stores them through a database function.

Table 10. Key architectural decisions

Decision	Reason
No custom application server	Supabase exposes the database safely through RLS and functions, so a separate API would add code and failure points without adding protection
All money logic in PostgreSQL functions	Functions run atomically next to the data and cannot be bypassed by a modified client
One money engine (`_transfer_core`)	Transfers, bill payments, request payments and scheduled runs share one tested code path for locking, limits and ledger writes
Balances derived from the ledger	There is no balance field that could drift from the transaction history
Integer centavos	Avoids floating-point rounding errors
Idempotency key on every money request	Retries and double taps return the first result instead of moving money twice
Row Level Security on every table	Access rules apply to every client, including ones that call the API directly
Only the public anon key in clients	The privileged service-role key never reaches a phone or browser

3.4 Use Case Diagrams

The use case diagrams identify what each actor can do. Figure 2 covers the customer. Transfers, bill payments and paying a money request all include a confirmation step with biometrics or the account password. Figure 3 covers the administrator and the automated scheduler.

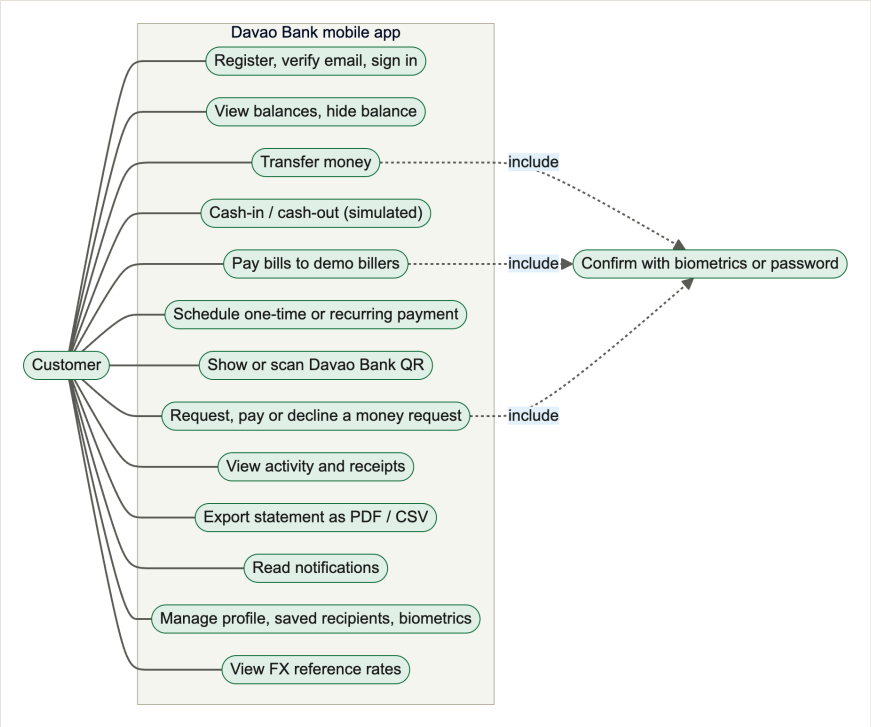


Figure 2. Use case diagram: customer

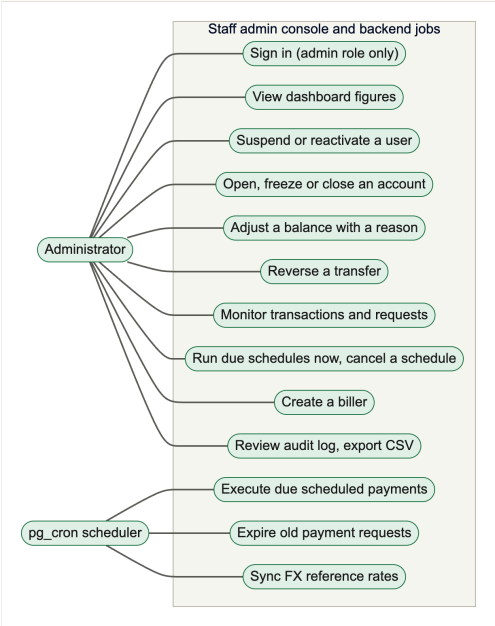


Figure 3. Use case diagram: administrator and scheduled jobs

3.5 Data Flow Diagrams

Figure 4 is the context diagram (level 0). It treats Davao Bank as a single process and shows the data exchanged with its external entities: customers, administrators, the Frankfurter exchange-rate service and the pg_cron scheduler.

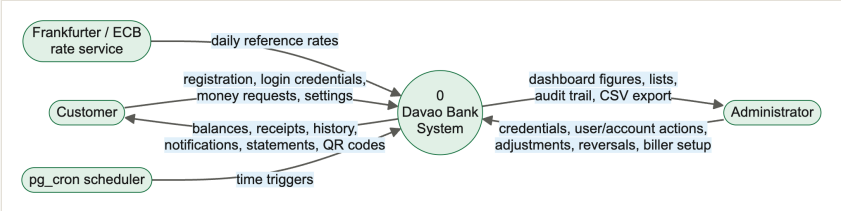


Figure 4. Context diagram (DFD level 0)

Figure 5 decomposes the system into seven processes and nine data stores. Process 2.0, "Process money movements", is the money engine; processes 3.0 (scheduling) and 4.0 (payment requests) do not move money themselves but call the engine. Process 6.0, "Administer", writes to the audit log and reads from it for the audit viewer.

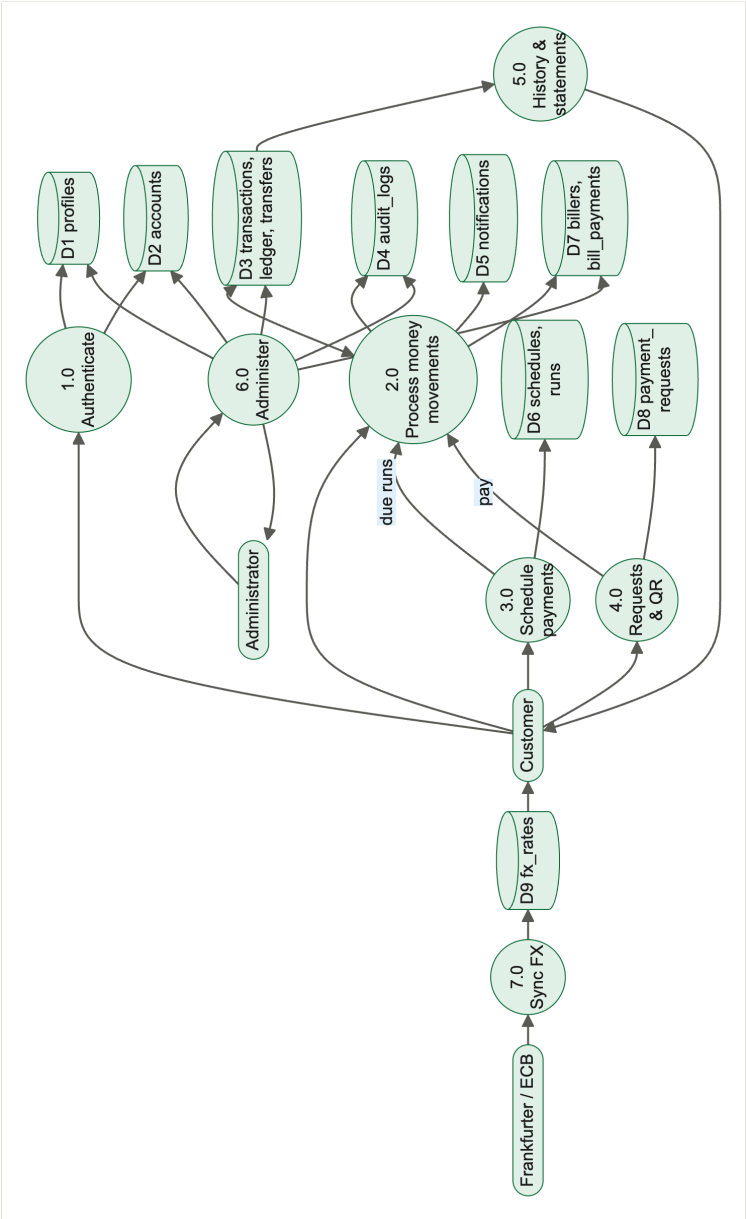


Figure 5. Data flow diagram level 1 (rotated to fit the page)

3.6 Database Design and Entity Relationship Diagrams

The database was designed first and the screens second. It has 15 tables in the `public` schema, two views and 39 functions. Figure 6 shows the core banking entities and Figure 7 the entities added in Phase 5 for recipients, billers, schedules, payment requests and exchange rates. Appendix A gives the full data dictionary.

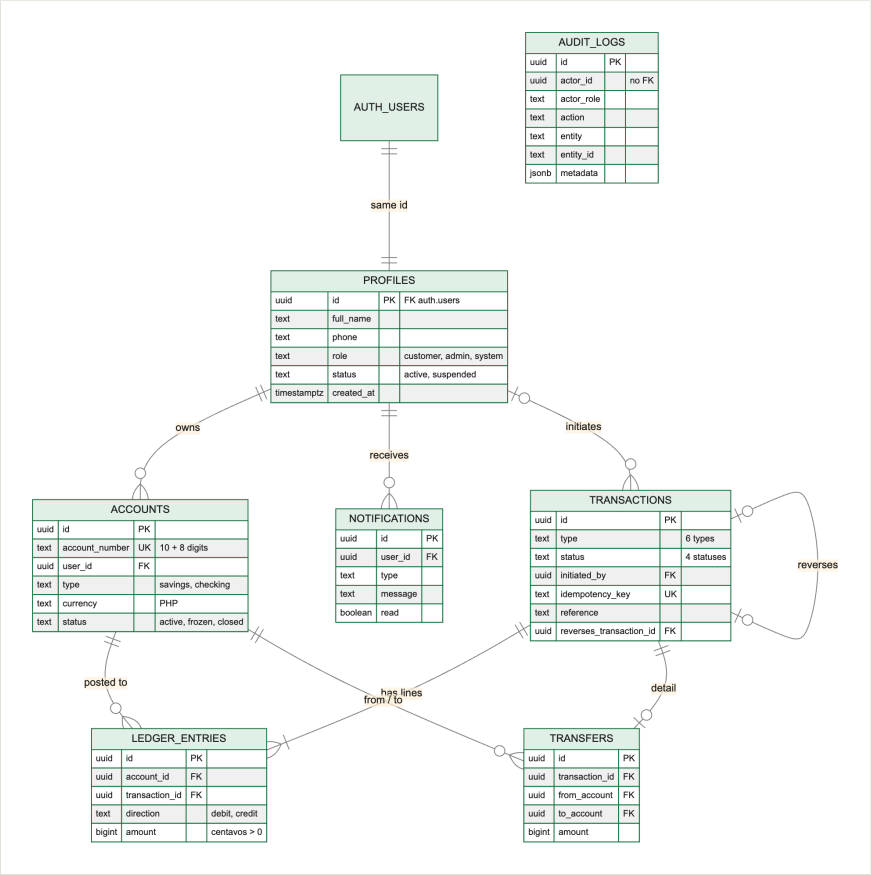


Figure 6. Entity relationship diagram: core banking entities

Every user who signs up has one row in `profiles`, keyed by the same identifier as the Supabase `auth.users` record, and one or more rows in `accounts`. A money event is recorded as one row in `transactions` (the envelope) and one or more rows in `ledger_entries` (the lines). Each ledger entry carries a positive amount in centavos and a direction, debit or credit. For transfers, a `transfers` row records the source and destination accounts. A reversal is a new transaction that points to the transaction it reverses; a unique index ensures that a transaction can be reversed only once.

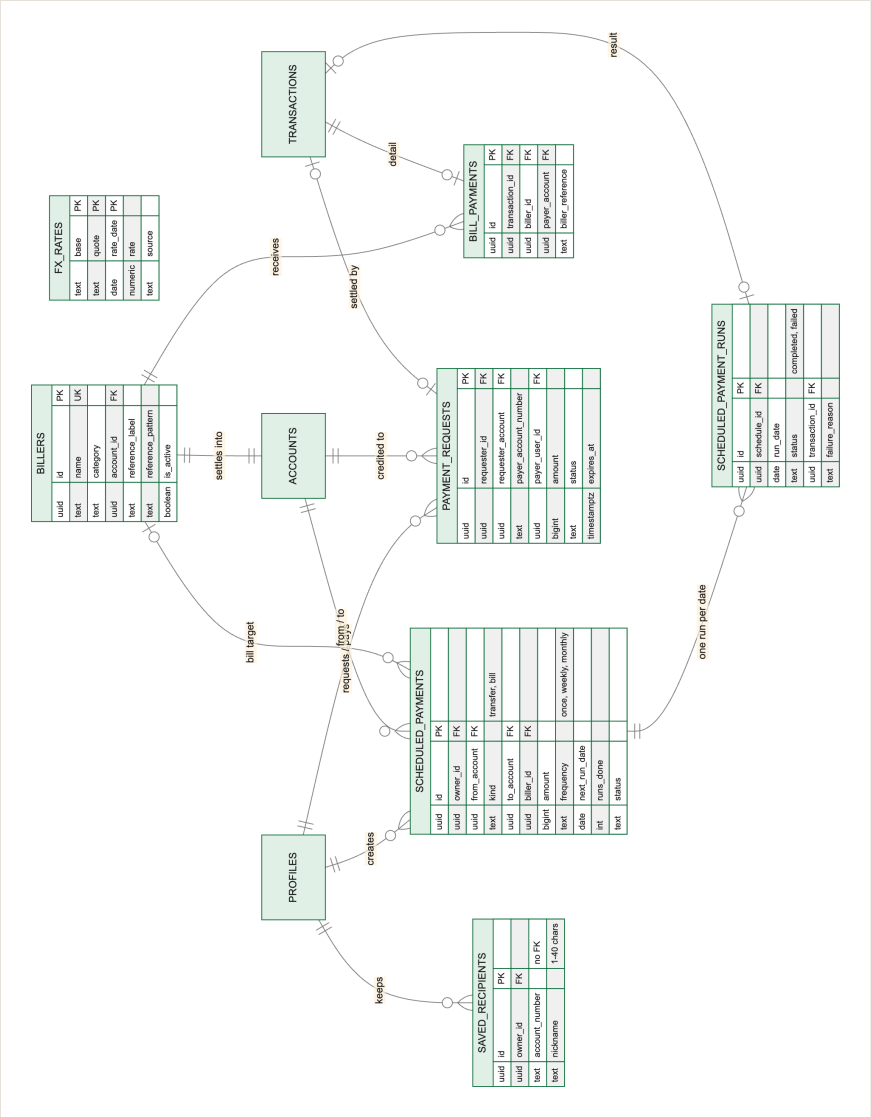


Figure 7. Entity relationship diagram: features added in Phase 5 (rotated to fit the page)

Each demonstration biller has its own settlement account, owned by the non-login `system` profile, so a bill payment is an ordinary balanced transfer from the customer to that account plus a `bill_payments` detail row. Scheduled payments are instructions; each execution is recorded once per date in `scheduled_payment_runs`. Payment requests record who asked whom for how much, and link to the transaction that paid them. The `fx_rates` table stands alone and is used only for display.

Table 11. Summary of database tables

Table	Purpose
profiles	One row per user: name, phone, role (customer, admin, system) and status (active, suspended)
accounts	Customer and biller settlement accounts: 10-digit number, type, currency, status
transactions	Envelope of every money event: type, status, initiator, idempotency key, reference
ledger_entries	Debit and credit lines; the source of truth for balances
transfers	Source and destination accounts of a transfer-type transaction
audit_logs	Append-only record of sensitive actions
notifications	In-app messages to users

Table	Purpose
saved_recipients	A customer's saved destination accounts and nicknames
billers	Demonstration billers with reference format rules
bill_payments	Biller and reference for each bill-payment transaction
scheduled_payments	One-time and recurring transfer or bill instructions
scheduled_payment_runs	One execution record per schedule per date
payment_requests	Money requests between customers
fx_rates	Daily ECB reference rates for USD, EUR and JPY against PHP
health_check	Connectivity probe used by tests and development diagnostics

3.7 User Interface Design

3.7.1 Design Principles

The interface design follows three principles. **Clarity over decoration:** money amounts, accounts and primary actions must be easy to find. **Confirmation before commitment:** every outgoing money movement passes through a review screen and an authorization step. **Local identity:** the palette, greeting and planned illustrations reflect Davao City without imitating any existing bank.

3.7.2 The Kadayawan Visual Identity

The color palette was sampled from the Kadayawan 2026 festival identity and approved on 23 October 2026. Mt. Apo green is the single primary action color; the other festival colors are used as accents for action tiles, icons and charts. The tokens are defined once in the shared package and used by both clients, and a check script confirms that the admin console's CSS variables match them.

Table 12. Core color tokens

Token	Value	Use
primary	#0C6D38	Primary buttons, balance card, active tab, links
green	#03AA56	Accent; Cash-in tile
blue	#0099C8	Accent; Transfer tile
orange	#FD4B12	Accent; Pay Bills and Cash-out tiles
yellow	#FEAD0B	Accent; Scheduled tile
red / redText	#EF2D23 / #C8102E	Destructive actions and error text
ink	#1A1A1A	Primary text
canvas	#F7F5F0	Screen background

The typeface is Plus Jakarta Sans, loaded in four weights. Spacing follows a 4-point scale (4, 8, 12, 16, 20, 24, 32, 40, 48), and corner radii are 10, 16, 24 and 32 points. Icons come from the Lucide set on both clients.

3.7.3 Navigation Structure

Figure 8 shows how a user moves through the mobile app. On launch the app restores the saved session and applies three gates in order: signed in, email verified and not suspended. A user who passes all three reaches the four tabs: Home, Activity, Inbox and Profile. Feature flows such as Transfer and Pay Bills open as bottom sheets over the current tab, so the user never loses his or her place.

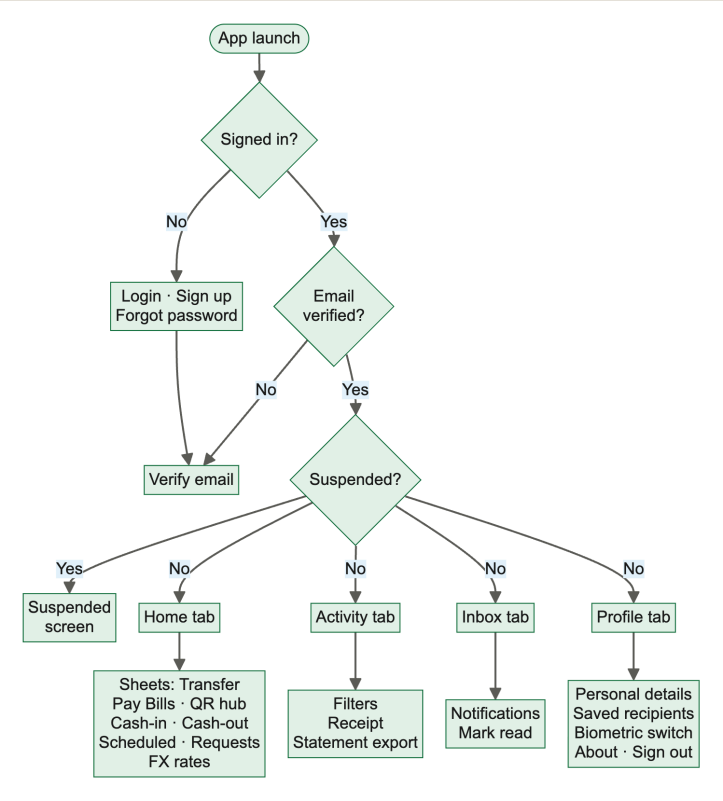


Figure 8. Mobile app navigation and screen flow

3.7.4 Principal Screens

Figure 9 shows the sign-in screen and the Home screen as they appeared on the iOS simulator on 3 November 2026. The Home screen places the balance card first, followed by six action tiles, a currency watch card, upcoming payments and recent activity. The blue gear button visible at the top right of mobile screenshots belongs to the Expo Go developer menu, not to the application.

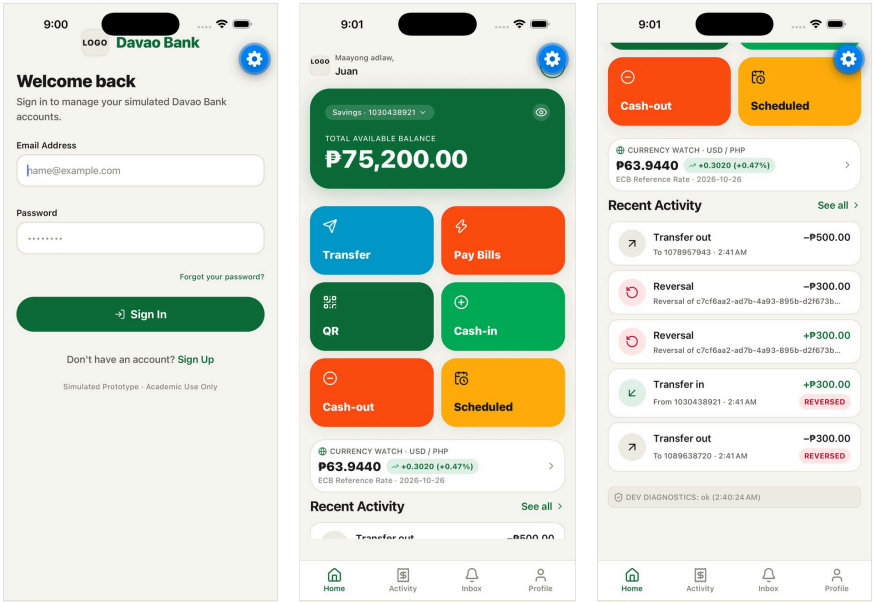


Figure 9. Sign-in screen, Home screen, and Home screen scrolled to recent activity (iOS simulator, 3 Nov 2026)

3.7.5 Accessibility Considerations and Known Gaps

Several accessibility measures are in place: balance hiding with screen-reader labels, haptic feedback on money results, Reduce Motion support in skeleton loaders and sheets, and an accessible summary for the exchange-rate chart. A user-interface audit on 3 November 2026 also found gaps that remain open and are scheduled for the next phase:

- The Plus Jakarta Sans font files load, but the text style refers to a font name that is not registered, so most text renders in the system font.
- White labels on the blue, green and orange action tiles have a contrast ratio of about 3.0 to 3.4 to 1, below the 4.5 to 1 required by WCAG AA for text of that size. The muted grey text color is also below 4.5 to 1.
- Several touch targets are smaller than 44 points, and some buttons lack screen-reader roles and labels.

Chapter 4. System Implementation

This chapter documents the system as it exists in the repository on 4 November 2026 (branch `phase-6-hardening`, commit `f5f3086`). Where Phase 6 Stage B security work was still uncommitted, it is identified as in progress. Line-level references to source files were verified during the review that produced this document.

4.1 Application Structure

The code is organised as an npm workspaces monorepo. Each client is a workspace, the shared code is a third workspace, and the database is defined entirely by SQL migration files that are applied in order and never edited after the fact.

```
davao-bank/
├── apps/
│   ├── mobile/              Customer app (Expo, React Native)
│   │   ├── App.tsx          Session restore, gates, tab navigator
│   │   ├── assets/          App icons, illustrations, biller icons
│   │   └── src/
│   │       ├── components/  Button, Card, BottomSheet, TextField, AmountInput, ...
│   │       ├── context/     SettingsContext (hide balance, biometrics, confirmation)
│   │       ├── lib/         Supabase client configuration
│   │       └── screens/     Tabs and feature sheets (Transfer, PayBills, QR, ...)
│   └── admin/               Staff console (Next.js App Router)
│       └── src/
│           ├── middleware.ts      Session refresh and admin gate
│           ├── app/               One folder per page; actions.ts (server actions)
│           ├── components/ui/     Table, Drawer, ConfirmDialog, StatTile, ...
│           └── lib/               Server and browser Supabase clients
├── packages/shared/         Supabase wrappers, types, tokens, currency/QR/error utils
├── supabase/
│   ├── migrations/          12 SQL migrations (schema, policies, functions, jobs)
│   ├── functions/fx-sync/   Edge Function for exchange rates
│   └── seed.sql             Local seed data
├── scripts/                 Seed script, live test suites, PGlite review harness
└── e2e/                     Maestro (mobile) and Playwright (admin) flows
```

4.2 Customer Mobile Application

4.2.1 Navigation and Screen Organisation

`App.tsx` decides what to render without a navigator: a splash screen while fonts and the session load, the authentication screens when signed out, the email verification screen when the email is unconfirmed, the suspended screen when the profile is suspended, and otherwise the main app. The main app is a bottom tab navigator with four tabs: Home, Activity, Inbox (with an unread badge) and Profile. Every feature flow is a React Native modal presented as a bottom sheet by the shared `BottomSheet` component and opened by a state flag in its parent screen. The exchange-rate screen is the exception; it is a full page sheet.

4.2.2 State Management

The app does not use a global store. Each screen holds its own state with React hooks and loads the data it needs. One React context, `SettingsContext`, provides settings that several screens share: whether balances are hidden, whether biometric confirmation is enabled, what kind of biometrics the device supports, and the confirmation function used before money moves. These preferences are stored on the device in AsyncStorage under the keys `@davao_bank_hide_balance` and `@davao_bank_biometric_enabled`. The Supabase session is also stored on the device; Phase 6 Stage B is replacing plain storage with an encrypted store whose key is kept in the device keychain through `expo-secure-store`.

4.2.3 Connection to Supabase

The mobile client is created by a factory in the shared package and configured with AsyncStorage for the session, automatic token refresh and no URL-based session detection. It is given only the public anon key, read from the environment variables `EXPO_PUBLIC_SUPABASE_URL` and `EXPO_PUBLIC_SUPABASE_ANON_KEY`. Every request also carries the signed-in user's token, so the database knows who is calling and applies Row Level Security accordingly.

4.2.4 Confirmation Before Money Moves

Every outgoing money movement except cash-in calls `confirmMoneyAction` from `SettingsContext` before contacting the server. If biometrics are enabled and available, it asks for Face ID or a fingerprint; the device passcode is accepted as a fallback by the operating system. If biometrics are off, unavailable or fail, it shows a password dialog and checks the password by signing in again with the user's own email. The function never returns success unless one of these checks passes, a rule added after the Phase 5 review found that devices without biometrics skipped confirmation entirely. Cancelling returns a "cancelled" result, and the flow shows that nothing was sent.

4.2.5 Error Handling

Money operations pass server errors through `toUserMessage`, a shared function that converts database messages into plain language. For example, "Insufficient funds" becomes "Not enough balance. You have ₱X available." and any unrecognised error becomes "Something went wrong. No money was sent. Try again." The original message is logged in development builds only. Errors appear in an inline red card on the relevant step. Some secondary screens, including sign-in and statement export, still show the raw message; this is listed in Section 5.6.

Bottom sheets protect money flows from accidental dismissal. While a request is being processed the sheet cannot be closed. On later steps, tapping outside the sheet goes back one step; on the first step with input it asks "Discard this transfer? Your details will be lost." before closing. Non-money sheets still close immediately on an outside tap.

4.3 Detailed Feature Documentation: Mobile Application

Each function below is described with the same ten fields: purpose, user, entry point, process, inputs, validation, system processing, database interaction, output and exceptions. Amounts are entered in pesos and converted to integer centavos before they are sent. Limits quoted are those enforced by the database.

4.3.1 App Launch and Session Restore

Purpose	Restore a saved session and route the user to the correct screen.
User	Anyone opening the app.
Entry point	Starting the app.
Process	The app loads the Plus Jakarta Sans fonts and shows a splash screen, reads the stored session with `getSession`, and subscribes to authentication changes. If a user is signed in, it loads the profile; if no profile exists, it calls `ensure_my_profile` to create one and reloads. It then applies the gates: not signed in leads to Login, unverified email to Verify Email, suspended profile to the Suspended screen, otherwise the tabs.
Inputs	None.
Validation	None on the client; the gates depend on the session and profile.
System processing	supabase-js restores and refreshes the token. `ensure_my_profile` (SECURITY DEFINER) creates a customer profile and a savings account for the caller only if they are missing.
Database interaction	Reads `profiles`; may insert into `profiles` and `accounts`.
Output	The appropriate screen.
Exceptions	Profile load errors are logged and the tabs show placeholder names. Suspension is re-checked on sign-in events and Home reloads, not continuously.

4.3.2 Registration (Sign Up)

Purpose	Create a customer login and a first account.
User	A new customer.
Entry point	Login screen, "Sign Up" link.
Process	The user completes the form and taps "Create Account". After client validation the app calls `supabase.auth.signUp` with the name and phone as metadata, then shows the Verify Email screen.
Inputs	Full legal name; mobile number (optional); email address; password; password confirmation.
Validation	Client: name and email required, password at least 6 characters, passwords match. Server: Supabase Auth checks email format and uniqueness.

System processing	The trigger `on_auth_user_created` runs `handle_new_user`, which inserts a customer profile and a PHP savings account with a generated 10-digit number beginning with "10".
Database interaction	Inserts into `auth.users`, `profiles` and `accounts`.
Output	The Verify Email screen.
Exceptions	Supabase errors are shown as returned. The phone format is not validated. Phase 6 Stage B raises the password minimum to 10 characters.

4.3.3 Email Verification

Purpose	Ensure the email belongs to the user before the app can be used.
User	A newly registered or unverified customer.
Entry point	Automatically after sign-up, after a sign-in rejected with "email not confirmed", or the launch gate.
Process	The screen shows the email address. "Resend Verification Email" calls `supabase.auth.resend`; "Back to Sign In" returns to Login, signing out first if needed. After confirming through the email link, the user signs in again.
Inputs	None; the email is passed in.
Validation	Resend is ignored if no email is known.
System processing	Supabase Auth sends the email and records confirmation.
Database interaction	`auth.users` (managed by Supabase).
Output	A confirmation message, or the error returned.
Exceptions	The email link does not open the app; the user must return and sign in.

4.3.4 Sign In, Password Reset and Sign Out

Purpose	Authenticate a customer, request a password reset, and end the session.
User	A registered customer.
Entry point	Login screen; "Forgot your password?" link; Profile, "Sign Out".
Process	Sign in calls `signInWithPassword`. If the email is unconfirmed, the app routes to Verify Email. Forgot password calls `resetPasswordForEmail`. Sign out asks for confirmation, calls `signOut` and returns to Login.
Inputs	Email and password; email only for reset.
Validation	Both fields required for sign in; email required for reset. Supabase Auth checks the credentials.
System processing	Supabase Auth issues or revokes the session. On success the authentication listener re-applies the launch gates.
Database interaction	`auth.users`; `profiles` after sign-in.
Output	Home, Verify Email or Suspended screen; a "reset instructions sent" message; the Login screen after sign out.
Exceptions	The reset link opens Supabase's default site because no in-app reset screen exists, so the reset cannot be completed in the app. Device preferences are not cleared at sign out.

4.3.5 Home Dashboard and Balance

Purpose	Show balances, shortcuts and recent activity at a glance.
User	A verified, active customer.
Entry point	Home tab.
Process	On load and on pull-to-refresh the screen loads, in parallel, the profile, account balances and transaction history, then counts pending payment requests and the next three active schedules. It shows a greeting ("Maayong adlaw,"), the balance card with an account selector and an eye button that hides amounts, a banner for pending requests, six action tiles (Transfer, Pay Bills, QR, Cash-in, Cash-out, Scheduled), the USD/PHP currency watch card, upcoming payments and the five most recent transactions.
Inputs	None; taps on tiles and cards.
Validation	None.
System processing	Balances come from the `account_balances` view, which sums ledger credits minus debits under the caller's RLS. History comes from the `get_account_transactions` function.

Database interaction	Reads `profiles`, `account_balances`, `get_account_transactions`, `payment_requests`, `scheduled_payments` and `fx_rates`.
Output	The dashboard (Figure 9).
Exceptions	Load failures are logged silently. The balance card label reads "Total Available Balance" and showed the total of all accounts while naming one account (observed 3 Nov 2026; corrected in Phase 6 Stage 0). If no exchange-rate data loads, the currency card shows built-in placeholder values.

4.3.6 Transfer

Transfer is the central money function. Figure 10 shows its three input steps, and Figure 27 in Section 4.7 shows the request it sends.

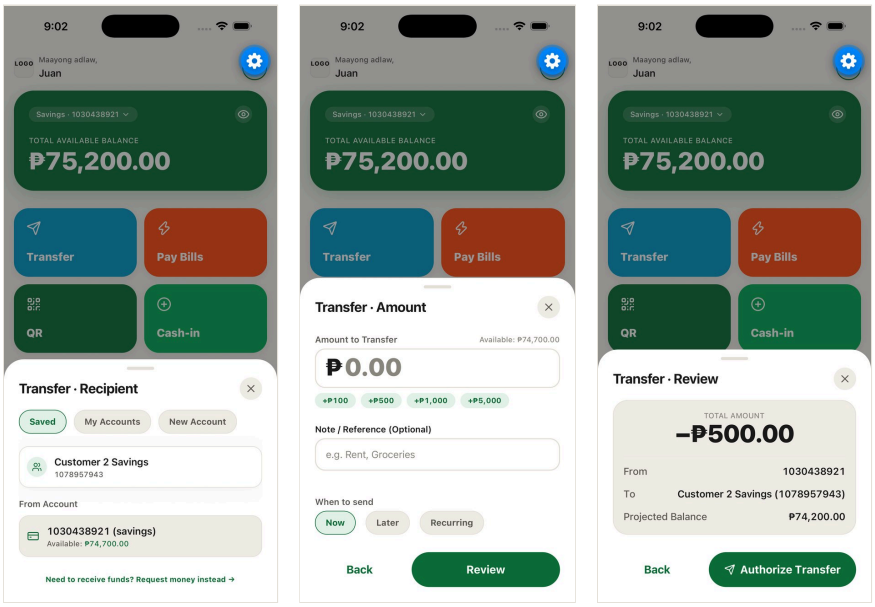


Figure 10. Transfer: recipient, amount and review steps (iOS simulator, 3 Nov 2026)

Purpose	Move pesos to another Davao Bank account, now, once later, or on a recurring schedule.
User	An active customer.
Entry point	Home, "Transfer" tile; also opened pre-filled after scanning a QR code.
Process	1) Recipient step: choose a saved recipient, one of the user's own accounts, or enter a new 10-digit number (optionally saving it with a nickname). 2) Amount step: amount with preset chips, optional note, and timing (Now, Later or Recurring with weekly or monthly frequency and an optional number of runs). 3) Review step showing amount, from, to, note, timing and projected balance. 4) Authorize with biometrics or password. 5) Submit. 6) Result screen.
Inputs	Recipient account number or selection; amount; note (up to 60 characters); timing; start date (YYYY-MM-DD); frequency; maximum runs.
Validation	Client: recipient required, not the same account, amount greater than zero and not above the available balance. Server: authenticated and not suspended; amount at most ₱5,000,000; source owned by the caller and active; destination exists and is active; same currency; sufficient funds. For schedules: start date not in the past (Asia/Manila), at most 20 active schedules.
System processing	An idempotency key (prefix `transfer_`) is created when the sheet opens and regenerated whenever the recipient, amount, note or source changes. "Now" calls `transfer_by_account_number`, which resolves the number and calls `transfer`, which calls the engine `_transfer_core` (Section 4.5.1). "Later" and "Recurring" call `create_scheduled_transfer_to_number`, which resolves the number on the server and calls `create_scheduled_payment`. If "save recipient" was ticked, `add_saved_recipient` is called afterwards.
Database interaction	Now: inserts one `transactions` row, one `transfers` row, a debit and a credit in `ledger_entries`, an audit entry and two notifications. Scheduled: inserts a `scheduled_payments` row and an audit entry.
Output	"Transfer Sent" with recipient and transaction ID, or "Transfer Scheduled"; a replayed request shows "This payment was already processed".

Exceptions	Cancelled authorization shows that nothing was sent. Server errors are mapped to plain language. Known defects found on 4 Nov 2026: refreshing Home after success re-runs the sheet's reset logic, which replaces the result screen after about a second and, on the QR path, returns to the review step with a new key so that a second tap would send a second transfer. The "Updated Balance" on the receipt falls back to the pre-transfer value. These are listed in Section 5.6.

4.3.7 Cash-in (Simulated Deposit)

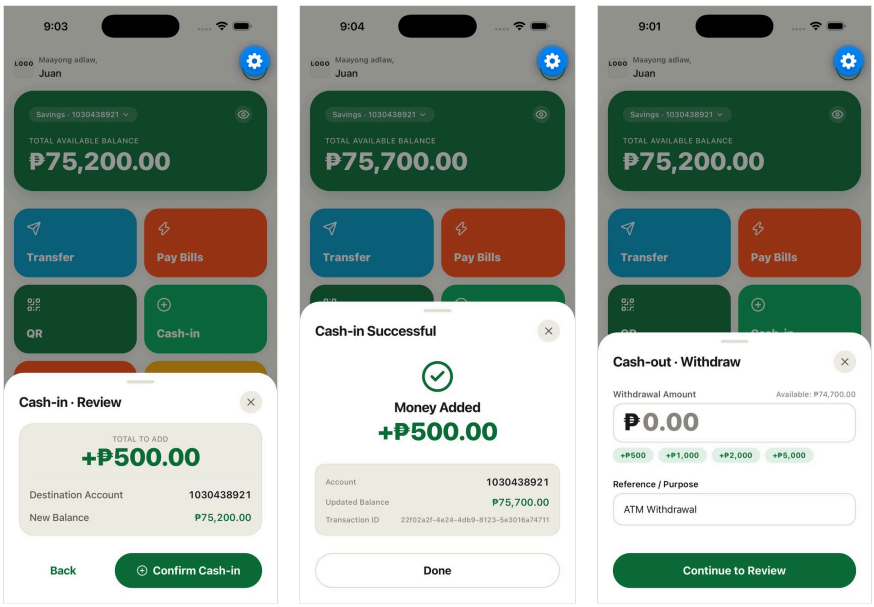


Figure 11. Cash-in review and result screens, and the Cash-out input step

Purpose	Add play money to an account for demonstration.
User	An active customer.
Entry point	Home, "Cash-in" tile.
Process	Enter an amount (presets ₱500 to ₱5,000) and a reference, review, tap "Confirm Cash-in", and see the result.
Inputs	Amount; reference (pre-filled "Online Top-up").
Validation	Client: amount greater than zero. Server: idempotency key required; authenticated and not suspended; amount at most ₱5,000,000; account owned by the caller and active.
System processing	`deposit` locks the account row, inserts the transaction and one credit entry, and writes the audit entry and notification. No biometric confirmation is required because no money leaves the user.
Database interaction	Inserts into `transactions` (type deposit), `ledger_entries` (one credit), `audit_logs` and `notifications`.
Output	"Money Added" with account, updated balance, date and transaction ID.
Exceptions	The deposit is single-sided: no contra account is debited (Section 4.5.2).

4.3.8 Cash-out (Simulated Withdrawal)

Purpose	Remove play money from an account for demonstration.
User	An active customer.
Entry point	Home, "Cash-out" tile.
Process	Enter an amount and a reference, review, authorize with biometrics or password, and see the result.
Inputs	Amount; reference (pre-filled "ATM Withdrawal").
Validation	Client: amount greater than zero and not above the balance. Server: as for cash-in, plus "Insufficient funds" when the locked balance is too low.
System processing	`withdraw` locks the account, computes the balance under the lock, and inserts the transaction and one debit entry.

Database interaction	Inserts into `transactions` (type withdrawal), `ledger_entries` (one debit), `audit_logs` and `notifications`.
Output	The result screen titled "Cash-out Prepared".
Exceptions	The title "Prepared" suggests the operation is pending although it is complete; Phase 7 replaces it with "Cash-out complete".

4.3.9 Pay Bills

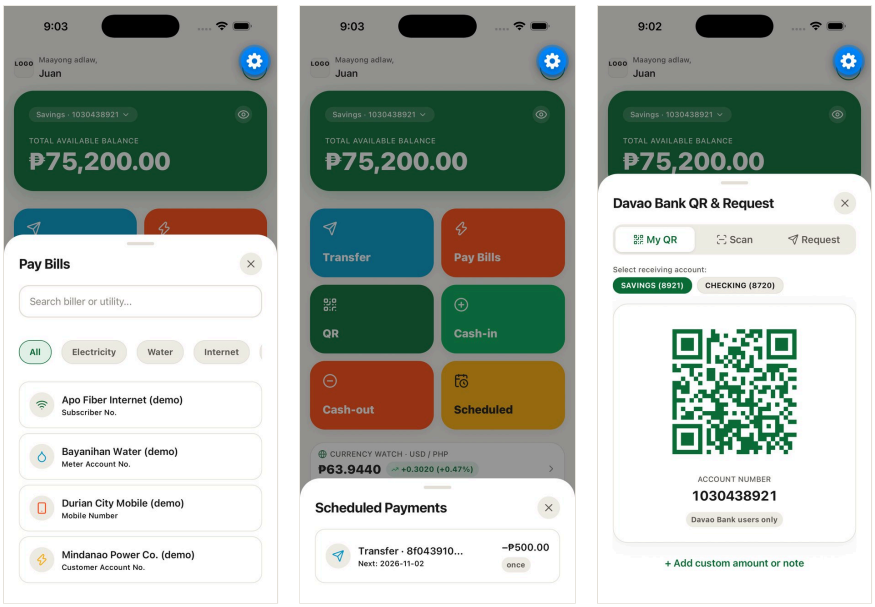


Figure 12. Pay Bills biller list, Scheduled Payments list, and the QR hub

Purpose	Pay a demonstration biller now, once later, or on a schedule.
User	An active customer.
Entry point	Home, "Pay Bills" tile.
Process	Six steps: choose a biller (search and category chips), enter the biller reference, enter the amount, choose timing, review, then authorize and pay. A result screen follows.
Inputs	Biller; reference (labelled by the biller, for example "Meter Account No."); amount; timing and date; frequency and maximum runs for recurring payments.
Validation	Client: reference required and matching the biller's pattern; amount above zero and within the balance. Server: biller exists and is active; reference matches the biller's regular expression; then all engine checks.
System processing	Now: `pay_bill` calls `_transfer_core` with type `bill_payment` to the biller's settlement account and inserts a `bill_payments` row. Scheduled: `create_scheduled_payment` with kind `bill`.
Database interaction	`transactions`, `transfers`, two `ledger_entries`, `bill_payments`, `audit_logs`, `notifications`; or `scheduled_payments`.
Output	"Payment Completed" with the first eight characters of the reference ID, or "Scheduled for" a date.
Exceptions	The reference helper text shows the raw pattern (for example `^[0-9]{10}$`). Billers are fictional and marked "(demo)".

4.3.10 Scheduled Payments

Purpose	List, inspect and cancel scheduled transfers and bill payments.
User	An active customer.
Entry point	Home, "Scheduled" tile or "See all" on upcoming payments; Profile, "Scheduled Payments".
Process	The list shows each schedule's recipient or biller, next date, amount and frequency. Tapping one shows details and the execution history. "Cancel Schedule" asks for confirmation, then for biometrics or password.
Inputs	Selection; confirmation.
Validation	Server: the caller owns the schedule (or is an administrator).

System processing	`list_my_scheduled_payments` returns the caller's schedules with account numbers and biller names; runs are read from `scheduled_payment_runs`; `cancel_scheduled_payment` sets the status to cancelled. Execution is described in Section 4.5.4.
Database interaction	Reads `scheduled_payments`, `accounts`, `billers`, `scheduled_payment_runs`; updates `scheduled_payments`; writes `audit_logs`.
Output	Updated list.
Exceptions	There is no edit screen although an update function exists. Errors appear in a system alert.

4.3.11 QR Hub: My QR, Scan and Request Money

Purpose	Receive money by QR code, pay by scanning, and request money from another customer.
User	An active customer.
Entry point	Home, "QR" tile; Transfer, "Request money instead".
Process	My QR shows a QR code for the selected account, optionally with an amount and note. Scan opens the camera, reads a code and opens Transfer pre-filled; it never sends money by itself. Request asks for the payer's account number, amount and note.
Inputs	Account selection, optional amount and note (QR); camera permission (Scan); payer account number, amount, note (Request).
Validation	QR payload: format `davaobank://pay?v=1&acct=<10 digits>[&amt;amt=<centavos>][&note=<text>]`, amount at most ₱500,000, note at most 60 characters; foreign or malformed codes show "This isn't a Davao Bank QR code". Request (server): amount at most ₱500,000; payer account exists, is active and is not the requester's; at most 10 requests per day.
System processing	QR payloads are built and parsed by shared functions. Requests call `create_payment_request`, which stores the request with a seven-day expiry and notifies the payer.
Database interaction	Request: inserts into `payment_requests`, `audit_logs`, `notifications`.
Output	A QR code; a pre-filled transfer; "Request Sent!"
Exceptions	Typing a QR amount above ₱500,000 causes an error during rendering. The request form clears if the sheet is closed by an outside tap.

4.3.12 Payment Requests

Purpose	Pay or decline incoming requests and track or cancel outgoing ones.
User	An active customer.
Entry point	Home pending-request banner; Profile, "Payment Requests".
Process	Incoming and Outgoing tabs. Pay Now authorizes with biometrics or password and pays; Decline and Cancel ask for confirmation first.
Inputs	Selection; confirmation.
Validation	Server: only the payer may pay or decline and only the requester (or an administrator) may cancel; the request must be pending and not expired; the source account must be the caller's and active.
System processing	`pay_payment_request` locks the request row, pays through `_transfer_core`, marks the request paid and links the transaction. A repeat with the same key returns the original result. `decline_payment_request` and `cancel_payment_request` change the status. A daily job expires requests older than seven days.
Database interaction	Updates `payment_requests`; on payment also `transactions`, `transfers`, `ledger_entries`, `audit_logs`, `notifications`.
Output	Updated lists and a status banner.
Exceptions	The source account is chosen automatically. The authorization prompt shows the requester's internal account identifier instead of a masked account number.

4.3.13 Activity and Transaction Receipt

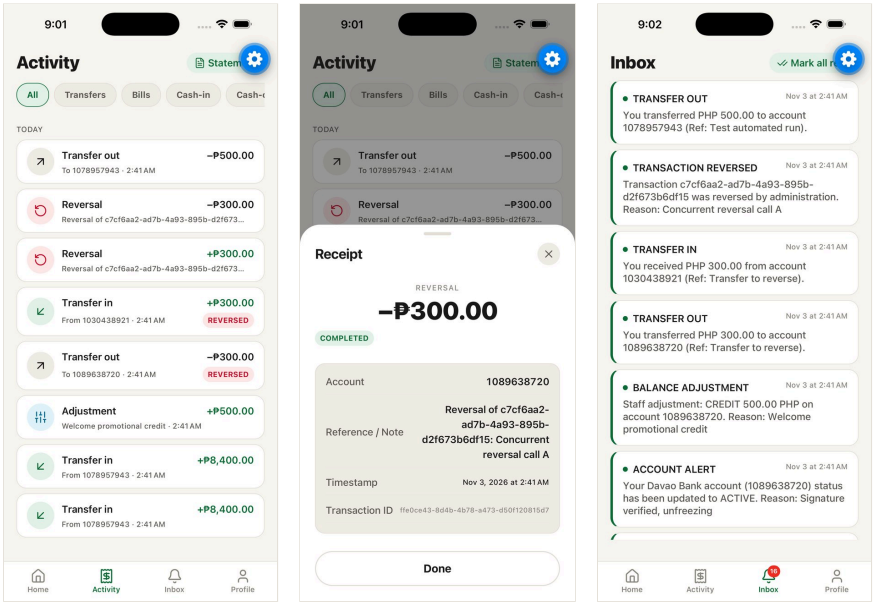


Figure 13. Activity tab, transaction receipt, and Inbox tab (iOS simulator, 3 Nov 2026)

Purpose	Show the full transaction history and a receipt for each item.
User	An active customer.
Entry point	Activity tab; Home, "See all".
Process	History is grouped by day ("TODAY", "YESTERDAY", then dates) and can be filtered by type: Transfers, Bills, Cash-in, Cash-out, Adjustments or Reversals. Tapping a row opens the receipt with type, amount, status, account, counterparty, reference, timestamp and transaction ID.
Inputs	Filter selection; row tap.
Validation	None.
System processing	`get_account_transactions` returns the caller's ledger lines with signed amounts and the counterparty account number.
Database interaction	Reads `transactions`, `ledger_entries`, `accounts`, `transfers` through the function, and `account_balances`.
Output	The history list and receipt (Figure 13).
Exceptions	Amounts in Activity are not masked when "hide balance" is on. Reversal rows show the internal identifier of the reversed transaction. There is no pagination.

4.3.14 Statement Export

Purpose	Produce a reconciled account statement as PDF or CSV.
User	An active customer.
Entry point	Activity, "Statement".
Process	Choose the account and a period (this month, last month, last three months or a custom range), then "Download PDF Statement" or "Export CSV Spreadsheet". The phone's share sheet opens with the file.
Inputs	Account; period or start and end dates.
Validation	Server: the caller owns the account; start date not after end date; period at most 366 days.
System processing	`get_account_statement` returns the opening balance, the period's transactions and the closing balance, so that opening plus movements equals closing. The PDF is built from HTML with `expo-print`; the CSV is written with `expo-file-system`; both are shared with `expo-sharing`. The PDF footer states that the bank is simulated.
Database interaction	Read only.
Output	A PDF or CSV file in the share sheet.

Exceptions	Errors are shown as returned. Dates must be typed in YYYY-MM-DD form.

4.3.15 Inbox Notifications

Purpose	Show in-app notices about money events and account changes.
User	An active customer.
Entry point	Inbox tab.
Process	Lists notifications newest first with type, time and message. Tapping an unread notice marks it read; "Mark all read" marks every notice read. The tab badge shows the unread count.
Inputs	Taps.
Validation	RLS allows users to read and update only their own notifications.
System processing	Notifications are created by database functions (Section 4.5.6); the app reads and updates them.
Database interaction	Reads and updates `notifications`.
Output	The list and badge (Figure 13).
Exceptions	The badge is empty until Inbox has been opened once. Message text is written by the database and uses "PHP 500.00" style amounts and administrator wording.

4.3.16 Profile, Saved Recipients and Settings

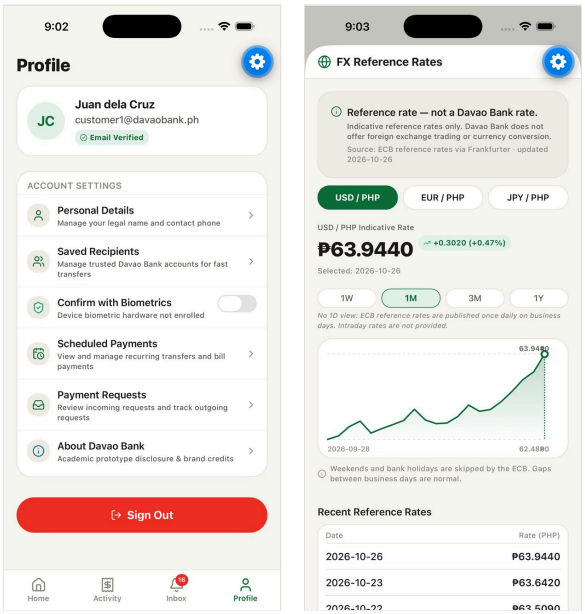


Figure 14. Profile tab and FX reference rates screen

Purpose	Manage personal details, saved recipients, biometric confirmation and other shortcuts.
User	An active customer.
Entry point	Profile tab; Home avatar.
Process	Personal Details edits name and phone. Saved Recipients lists, adds (through `add_saved_recipient`), renames and deletes recipients. The biometric switch turns confirmation on or off; turning it off requires a successful biometric check. About shows the prototype disclosure and brand credits. Sign Out ends the session.
Inputs	Name, phone; recipient number and nickname; switch state.
Validation	Name required. Recipients (server): nickname 1 to 40 characters; at most 50 recipients and 20 additions per day; account exists, is active and is not the user's own; no duplicates. RLS and a column grant allow customers to change only a recipient's nickname. A trigger blocks customers from changing their own role or status.
System processing	Profile update is a direct update under RLS; recipient additions use a SECURITY DEFINER function.
Database interaction	`profiles`, `saved_recipients`, `audit_logs`.
Output	Updated profile and lists.

Exceptions	Renaming a recipient uses an iOS-only dialog, so it does nothing on Android. The About screen still says "Phase 5 Build".

4.3.17 FX Reference Rates

Purpose	Show indicative ECB reference rates for context. Davao Bank does not exchange currency.
User	An active customer.
Entry point	Home, currency watch card.
Process	Choose USD, EUR or JPY against PHP and a range of one week, one month, three months or one year. The screen shows the latest rate with its change, a line chart that can be scrubbed, a disclaimer and a table of recent rates.
Inputs	Pair and range.
Validation	None.
System processing	Rates are read from `fx_rates`. The `fx-sync` Edge Function refreshes them daily (Section 4.5.5).
Database interaction	Reads `fx_rates`.
Output	The chart screen (Figure 14), with the disclaimer "Reference rate — not a Davao Bank rate."
Exceptions	When the daily sync is not running the latest date shown becomes stale; on 3 Nov 2026 the latest rate was dated 26 Oct 2026.

4.3.18 Suspended Account

Purpose	Block a customer whom staff have suspended.
User	A customer whose profile status is suspended.
Entry point	Automatic, through the launch gate or the Home screen.
Process	The screen explains that access to money operations is restricted and offers "Check Status Again" and "Sign Out".
Inputs	None.
Validation	None on the client.
System processing	Every money function independently rejects a suspended caller with "Your profile is suspended", so the block holds even if a client ignores the screen.
Database interaction	Reads `profiles`.
Output	The suspended screen.
Exceptions	When the suspension is detected from Home the tab bar remains visible, but money functions still refuse.

4.4 Staff Administration System

4.4.1 Architecture of the Console

The staff console is a Next.js 16 application using the App Router. Each page is a server component that creates a Supabase client bound to the staff member's session cookie, loads its data, and passes it to a client component that handles searching, filtering and dialogs in the browser. Every change is made through a **server action** in `app/actions.ts`, which calls a database function with the staff member's own token and then refreshes the affected pages. Because the token belongs to the staff member, the database records the real person in the audit log.

Access is controlled at three layers. **Middleware** (`middleware.ts`) refreshes the session on every request, sends signed-out visitors to `/login`, and sends signed-in users whose profile role is not `admin` to `/unauthorized`. **Database functions** used by the console each begin by calling `is_admin()`, which returns true only for an active administrator, and raise "Admin privilege required" otherwise. **Row Level Security** lets administrators read all rows only through policies that call `is_admin()`. The console never uses the privileged service-role key; a server-only module for it exists but is not imported anywhere.

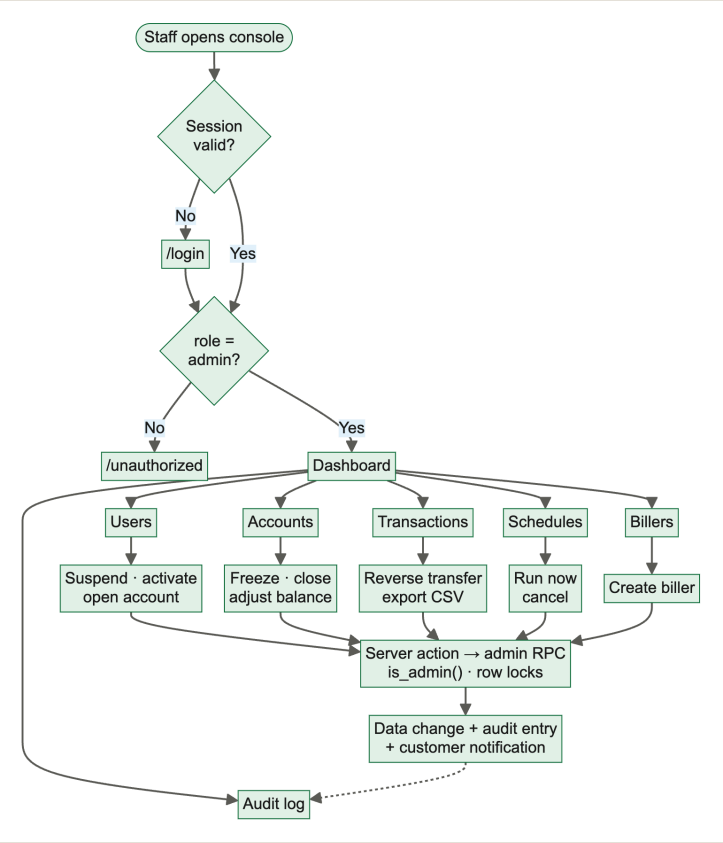


Figure 15. Administrative workflow in the staff console

Table 13. Admin console routes

Route	Page	Main data source
/login	Staff sign-in	Supabase Auth; `profiles.role`
/	Dashboard	`admin_get_dashboard_stats` and recent rows
/users	User and customer directory	`admin_get_users`, `account_balances`
/accounts	Deposit and ledger accounts	`account_balances`, `admin_get_users`
/transactions	Transaction oversight and reversals; payment requests	`transactions`, `ledger_entries`, `accounts`, `payment_requests`
/schedules	Scheduled and recurring payments	`scheduled_payments`, `scheduled_payment_runs`
/billers	Biller directory and settlement	`billers`, `accounts`
/audit	Audit trail	`audit_logs` (latest 100)
/api/export/transactions	CSV download	`transactions`, `ledger_entries`
/unauthorized	Access denied	None

4.4.2 Staff Sign-in

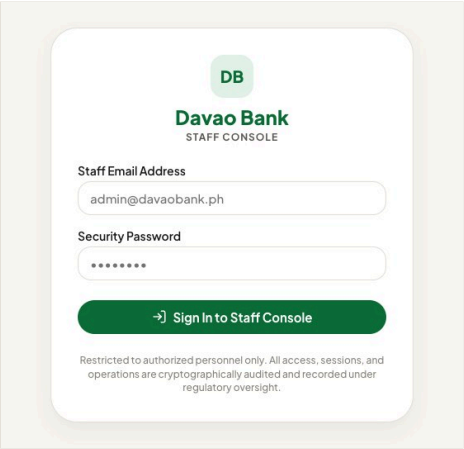


Figure 16. Staff console sign-in page

Purpose	Admit only active administrators to the console.
User	Bank staff with the administrator role.
Entry point	Any console address; unauthenticated visitors are redirected to `/login`.
Process	The staff member enters email and password and taps "Sign In to Staff Console". The page signs in with Supabase Auth, reads the profile role, and either opens the dashboard or signs out with "Access Denied".
Inputs	Staff email address; password.
Validation	Both fields required. Supabase Auth checks the credentials; the page and the middleware check `role = admin`.
System processing	The browser client stores the session in cookies so the middleware and server components can read it.
Database interaction	`auth.users`; reads `profiles.role`.
Output	The dashboard, or an error banner.
Exceptions	The middleware checks the role but not the status, so a suspended administrator can open pages, but every database function and RLS policy refuses him or her and the pages show empty data. The footer text claims activity is "cryptographically audited", which overstates the audit log.

4.4.3 Dashboard

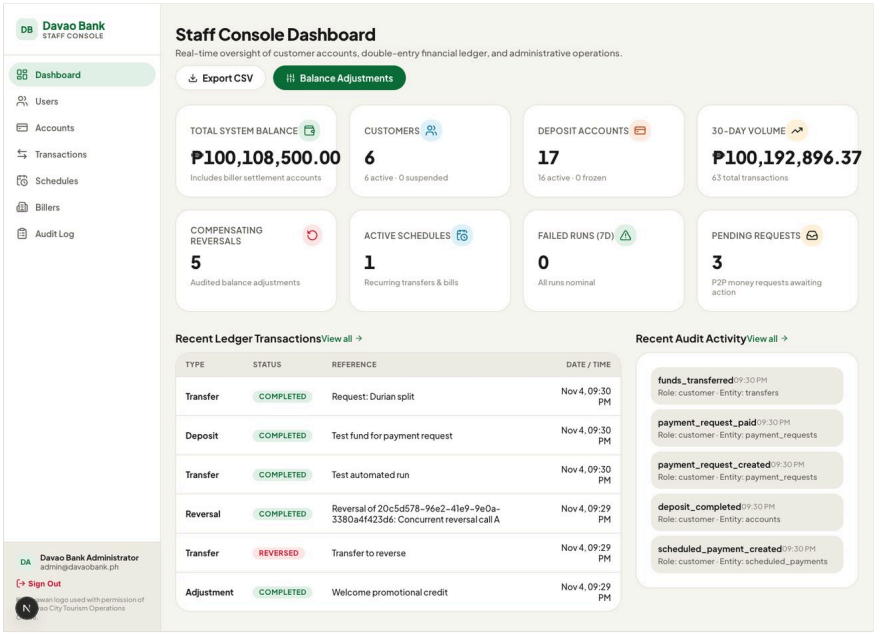


Figure 17. Staff console dashboard (local development server, 4 Nov 2026)

Purpose	Give staff an overview of the system.
User	Administrator.
Entry point	`/` or "Dashboard" in the sidebar.
Process	The page calls `admin_get_dashboard_stats` and three counting queries, then shows eight figures (total system balance, customers, deposit accounts, volume, compensating reversals, active schedules, failed runs in the last seven days, pending requests), the six most recent transactions and the five most recent audit entries. Buttons link to the CSV export and to account adjustments.
Inputs	None.
Validation	The function requires `is_admin()`.
System processing	The function counts profiles and accounts by status, sums all balances, counts transactions by status, sums all credit entries as volume and counts the last 30 days of transactions.
Database interaction	Reads `profiles`, `accounts`, `account_balances`, `transactions`, `ledger_entries`, `scheduled_payments`, `scheduled_payment_runs`, `payment_requests`, `audit_logs`.
Output	Figure 17.
Exceptions	Three known issues: the "30-day volume" figure actually shows all-time credits; the "failed runs" figure queries a column that does not exist and therefore always shows zero; and the "customers" count includes staff and the system profile.

4.4.4 User Management

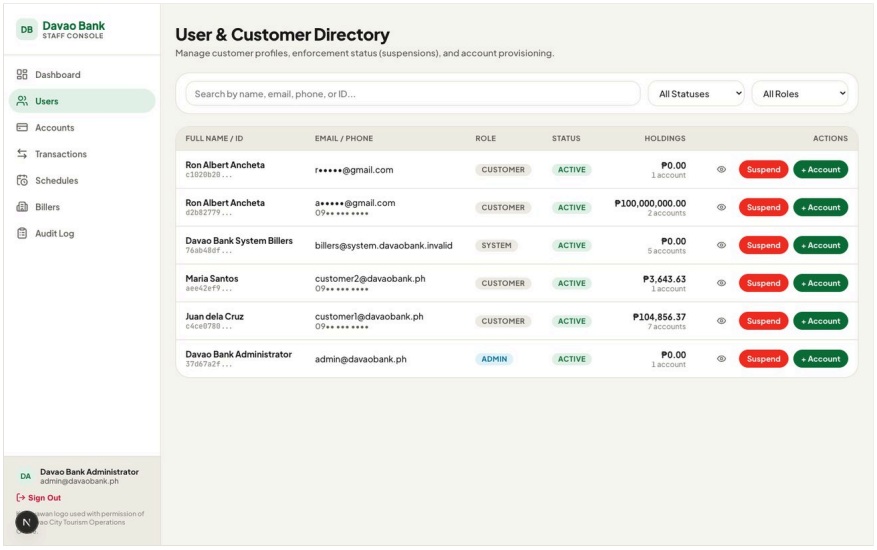


Figure 18. User and customer directory. Personal email addresses and phone numbers have been masked for this document.

Purpose	Find customers, view their accounts, suspend or reactivate them, and open additional accounts.
User	Administrator.
Entry point	Sidebar, "Users".
Process	Search by name, email, phone or identifier and filter by status (including "Needs attention" for logins without a profile) and role. The eye button opens a drawer with the user's accounts. "Suspend" or "Activate" opens a confirmation dialog that requires a reason. "+ Account" opens a drawer to choose savings or checking.
Inputs	Search text, filters, reason, account type.
Validation	Client: the reason cannot be empty. Server: `admin_set_user_status` requires an administrator, a valid status and a reason; `admin_open_account` requires an existing profile and a valid type.
System processing	`admin_set_user_status` updates the profile status (the profile guard trigger permits it because the caller is an administrator), writes `admin_user_status_changed` to the audit log and notifies the user. `admin_open_account` generates a unique account number, creates the account, audits and notifies.
Database interaction	Reads `auth.users` (through `admin_get_users`), `profiles`, `account_balances`; updates `profiles`; inserts `accounts`, `audit_logs`, `notifications`.
Output	Refreshed table; suspended customers are blocked on their next request (Figure 18).
Exceptions	There is no guard against suspending oneself or other administrators. Roles cannot be changed from the console.

4.4.5 Account Management and Balance Adjustment

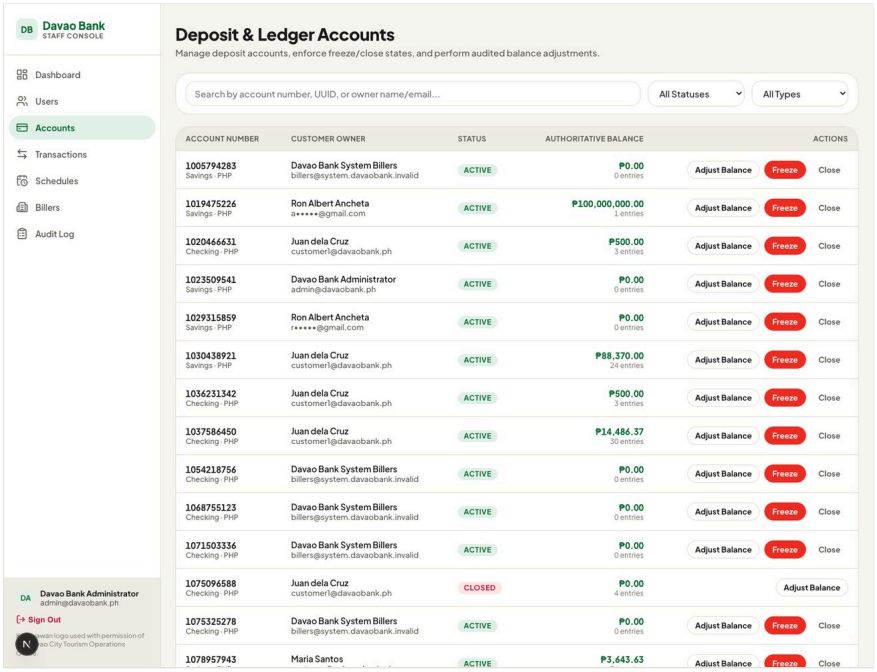


Figure 19. Deposit and ledger accounts. Personal email addresses have been masked.

Purpose	Freeze, unfreeze or close accounts, and correct balances with an audited adjustment.
User	Administrator.
Entry point	Sidebar, "Accounts"; dashboard, "Balance Adjustments".
Process	Search and filter accounts by number, owner, status and type. Freeze, Unfreeze and Close open a confirmation dialog with a mandatory reason. "Adjust Balance" opens a drawer showing the current balance, a credit or debit choice, an amount and a live preview of the resulting balance, then asks for a reason.
Inputs	Status action and reason; direction, amount and reason for adjustments.
Validation	Server, status: valid status; reason required; no change to the same status; closed accounts cannot change; an account can be closed only with a zero balance. Server, adjustment: idempotency key required; amount above zero; reason required; account not closed; a debit cannot make the balance negative.
System processing	Both functions lock the account row. `admin_adjust_balance` inserts an `adjustment` transaction whose reference is the reason and one ledger entry, records the balance before and after in the audit metadata, and notifies the owner. The idempotency key is generated when the drawer opens, so a double click cannot apply the adjustment twice.
Database interaction	Updates `accounts`; inserts `transactions`, `ledger_entries`, `audit_logs`, `notifications`.
Output	Refreshed balances (Figure 19).
Exceptions	There is no upper limit on adjustment amounts. The confirmation dialog for "Close" is mislabelled "Unfreeze Account". The adjustment notification truncates centavos.

4.4.6 Transaction Monitoring, Reversal and Export

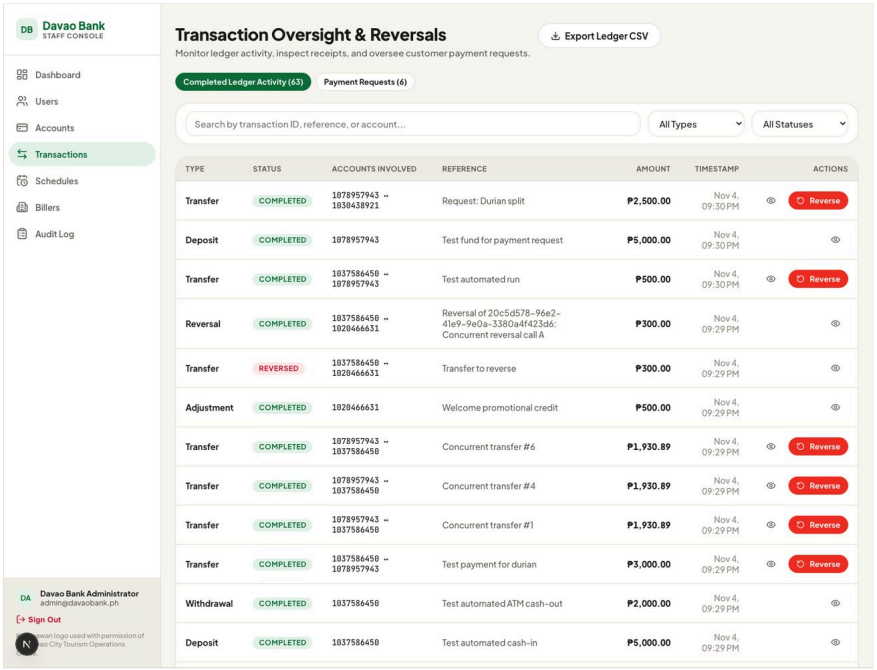


Figure 20. Transaction oversight and reversals

Purpose	Monitor all money movement, inspect receipts, reverse transfers, review payment requests and export data.
User	Administrator.
Entry point	Sidebar, "Transactions"; dashboard "View all"; "Export Ledger CSV".
Process	Two tabs list completed ledger activity and payment requests, with search and filters by type and status. A row opens a drawer with the transaction's amount, status, identifier, idempotency key and accounts. "Reverse" appears on completed transfers and opens a dialog requiring a reason. "Export Ledger CSV" downloads all transactions.
Inputs	Search text, filters, reason.
Validation	Server, reversal: administrator; idempotency key; reason; original must be completed, not already reversed and not itself a reversal; no affected account may go negative.
System processing	`admin_reverse_transaction` locks the original transaction first, then the affected accounts in a fixed order, inserts a `reversal` transaction pointing to the original, writes mirror-image ledger entries, marks the original "reversed", audits and notifies the owners (Figure 28). A unique index on the reversal reference makes a second reversal impossible even under concurrent attempts. The export route checks the administrator role and returns a CSV file.
Database interaction	Reads `transactions`, `ledger_entries`, `accounts`, `payment_requests`; on reversal inserts `transactions`, `ledger_entries`, `transfers`, `audit_logs`, `notifications` and updates the original transaction.
Output	Refreshed table (Figure 20) or a CSV download.
Exceptions	The type filter lacks "bill payment". The page and the export read whole tables without pagination, so results would be truncated beyond 1,000 rows. Exports are not audited.

4.4.7 Scheduled Payments, Billers and Audit Log

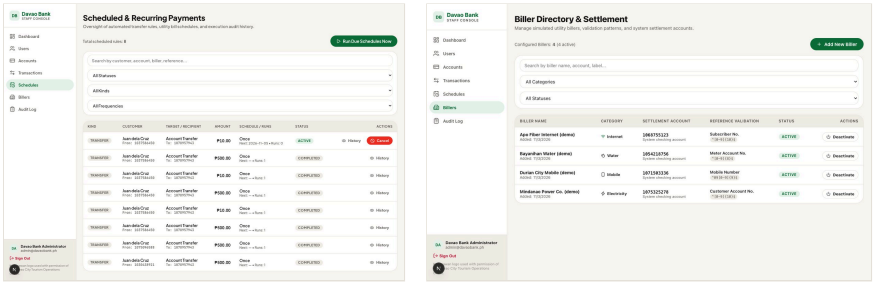


Figure 21. Scheduled payment oversight (left) and biller directory (right)

Purpose	Oversee customers' schedules, manage billers, and review the audit trail.
User	Administrator.
Entry point	Sidebar, "Schedules", "Billers" and "Audit Log".
Process	Schedules: search and filter all schedules, view run history, "Run Due Schedules Now", and cancel a schedule with a reason. Billers: list billers with their settlement account and reference rule, and "Add New Biller". Audit Log: the 100 most recent entries, filterable by action and role, with a drawer showing the full metadata.
Inputs	Filters; cancellation reason; biller name, category, reference label and pattern.
Validation	Server: each function requires an administrator. New billers need a unique name and a valid category; the client checks that the pattern is a valid regular expression.
System processing	`admin_run_due_schedules_now` audits the trigger and runs the scheduler immediately. `cancel_scheduled_payment` cancels and notifies the owner. `admin_create_biller` creates a checking settlement account owned by the system profile, then the biller, then an audit entry.
Database interaction	`scheduled_payments`, `scheduled_payment_runs`, `billers`, `accounts`, `audit_logs`, `notifications`.
Output	Figure 21 and Figure 22.
Exceptions	The biller Activate/Deactivate button updates the table directly instead of calling `admin_set_biller_active`; because billers have no update policy it silently changes nothing and writes no audit entry. A failed manual run is displayed in the success color. The audit view cannot reach entries older than the latest 100.

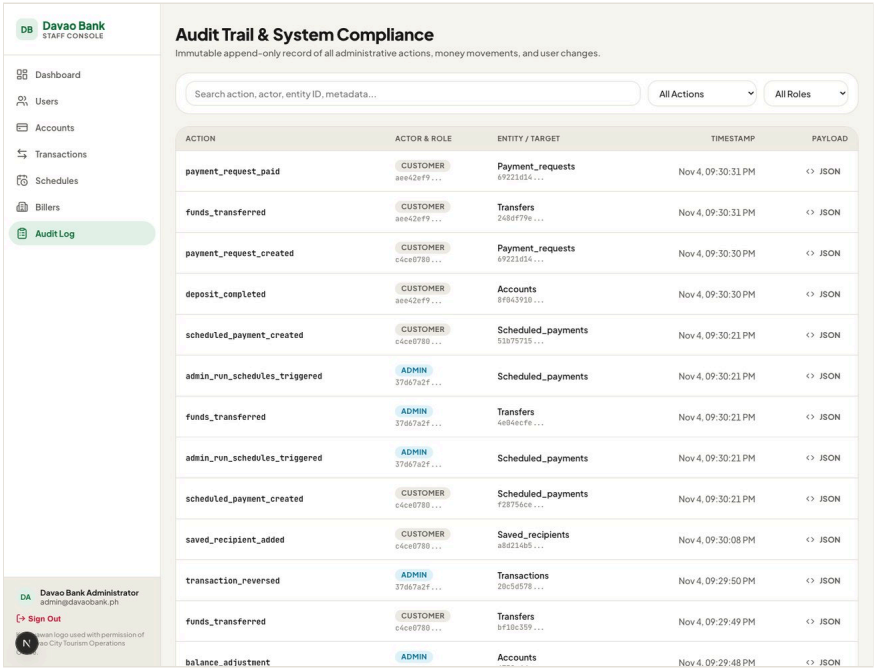


Figure 22. Audit trail

4.5 Backend and Database Operations

4.5.1 The Money Engine

All transfers, bill payments, payment-request payments and scheduled executions go through one internal function, `_transfer_core`. Clients cannot call it directly; they call thin wrappers that pass the caller's identity. Figure 23 shows the customer's path through a transfer, and the numbered steps below describe what the engine does inside one database transaction.

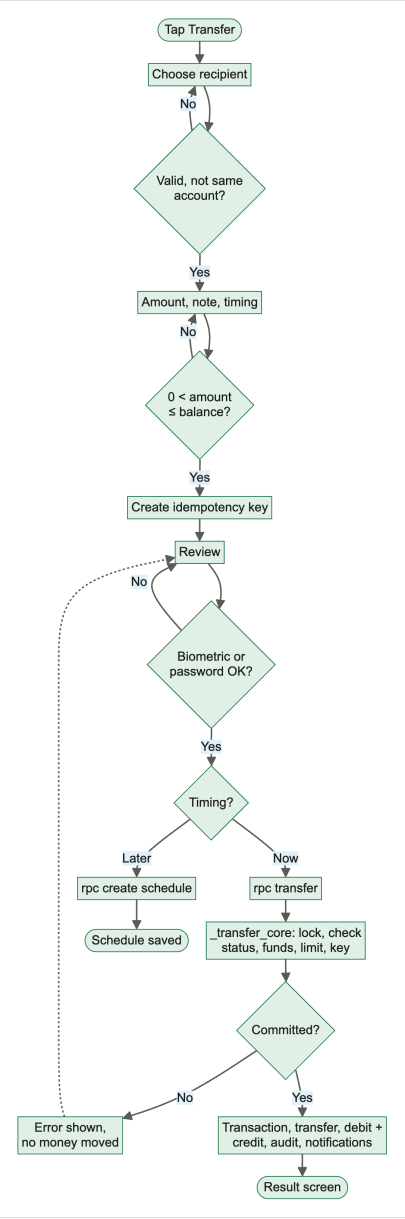


Figure 23. Activity diagram: transfer from the mobile app

1. Look up the idempotency key. If a transaction with this key exists, return it marked as a replay.
2. Check that the caller is signed in and the profile is active, the amount is above zero and at most ₱5,000,000, and the two accounts differ.
3. Lock both account rows with `SELECT ... FOR UPDATE`, always in the same order (lower identifier first) so that two opposite transfers cannot deadlock.

4. Confirm the source belongs to the caller and both accounts are active and in the same currency.
5. Compute the source balance from the ledger while holding the lock, and reject the request with "Insufficient funds" if it is too low.
6. Insert the transaction, the transfer row, a debit on the source and a credit on the destination.
7. Write the audit entry and the notifications, and return the transaction.

If any step fails, PostgreSQL rolls back everything, so no partial transfer can exist. If two requests with the same key race, the unique constraint on `idempotency_key` lets only one insert succeed; the other catches the violation and returns the winner's result.

4.5.2 Ledger Model

There is no balance column anywhere in the database. An account's balance is always the sum of its credit entries minus the sum of its debit entries, computed by the `account_balances` view or, under lock, inside the money functions. Table 14 shows what each operation writes.

Table 14. Ledger effect of each operation

Operation	Transaction type	Ledger entries	Other rows
Transfer	transfer	Debit source, credit destination	transfers
Bill payment	bill_payment	Debit payer, credit biller settlement account	transfers, bill_payments
Pay a request	transfer	Debit payer, credit requester	transfers; request marked paid
Scheduled run	transfer or bill_payment	As above	scheduled_payment_runs
Cash-in	deposit	One credit	—
Cash-out	withdrawal	One debit	—
Adjustment	adjustment	One credit or one debit	—
Reversal	reversal	Mirror of every original line	transfers (swapped); original marked reversed

Transfers, bill payments and request payments are balanced: their debits equal their credits. Cash-in, cash-out and adjustments are single-sided because the prototype has no internal "cash" or "suspense" account to take the other side. As a result the system as a whole does not net to zero; the difference equals the play money created by cash-in and credit adjustments minus that removed by cash-out and debit adjustments. This differs from the proposal's statement that every transaction nets to zero. The reconciliation tests therefore check two narrower invariants: every account's balance equals its ledger sum, and every transfer, bill payment and reversal nets to zero.

Transactions use four statuses. Functions only ever write `completed`, and a reversal changes the original to `reversed`. The statuses `pending` and `failed` are allowed by the table constraint but never written, because a failed request is rolled back entirely (Figure 24).

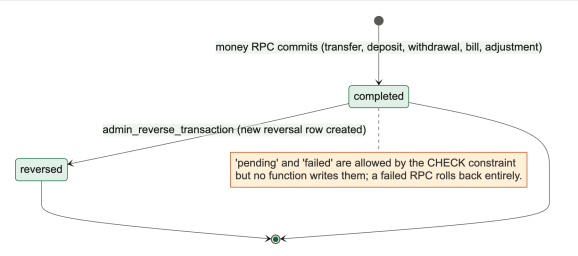


Figure 24. State diagram: transaction status

4.5.3 Limits and Business Rules

Table 15. Limits enforced by the database

Rule	Limit	Enforced by
Maximum single transfer, deposit, withdrawal, bill or scheduled amount	₱5,000,000	Money functions
Maximum payment request or QR amount	₱500,000	`create_payment_request`; QR utilities
Payment requests per day	10	`create_payment_request`
Payment request lifetime	7 days	Default `expires_at`; daily expiry job
Saved recipients	50 total; 20 additions per 24 hours	`add_saved_recipient`
Active scheduled payments	20 per user	`create_scheduled_payment`
Statement period	366 days	`get_account_statement`
Note or nickname length	60 / 40 characters	Table constraints and functions
Account closure	Balance must be zero	`admin_set_account_status`

4.5.4 Scheduled Payment Execution

A pg_cron job named `davao-run-schedules` runs every five minutes and calls `run_due_scheduled_payments`. Figure 25 shows its logic. The job first records a run row for the schedule and date; a unique constraint on that pair guarantees each schedule runs at most once per date even if two executions overlap. It then pays through the money engine as if the owner had initiated it, using the key `sched:<schedule id>:<date>`. A failure is recorded with its reason and the owner is notified, without affecting other schedules. Monthly schedules are calculated from the start date (start date plus n months), so a schedule starting on 31 January runs on 28 February and 31 March rather than drifting.

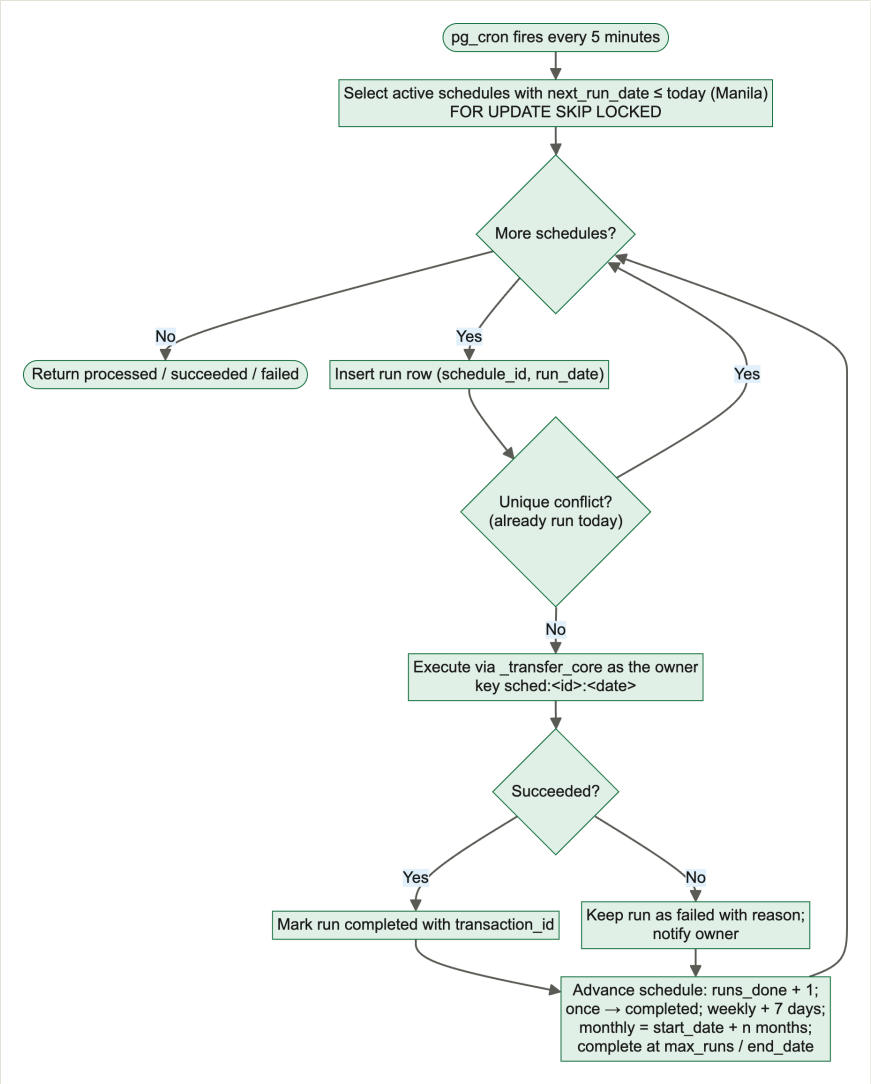


Figure 25. Activity diagram: scheduled payment executor

A failed run still advances the schedule and counts toward its maximum number of runs. This is a design choice that may surprise customers and is noted in Section 5.6.

4.5.5 Exchange-Rate Synchronisation

The `fx_rates` table was seeded with 768 European Central Bank reference rates (256 business days each for USD, EUR and JPY against PHP, from 24 October 2025 to 26 October 2026). The daily job `davao-daily-fx-sync` uses pg_net to call the `fx-sync` Edge Function at 04:00 UTC. The function accepts only requests carrying a header that matches the `FX_SYNC_SECRET` value; the job reads this secret from Supabase Vault at run time, so it never appears in the job definition. The function fetches the latest rates from the Frankfurter API and stores them through `upsert_fx_rates`. Rates are for display only and play no part in any money operation.

4.5.6 Notifications

Database functions insert notifications in the same transaction as the event they describe, so a notification exists only if the event committed. Table 16 lists them.

Table 16. Notification events

Event	Recipient	Type
Transfer, request payment or scheduled transfer	Sender and receiver	transfer_out, transfer_in
Bill payment	Payer	bill_payment
Cash-in / cash-out	Owner	deposit / withdrawal
Profile suspended or reactivated	User	account_alert
Account frozen, unfrozen or closed	Owner	account_alert
Account opened by staff	User	account_opened
Balance adjustment	Owner	balance_adjustment
Transaction reversed	Each affected owner	transaction_reversed
Schedule cancelled by staff	Owner	scheduled_payment_cancelled
Scheduled run failed	Owner	scheduled_payment_failed
Payment request received / paid / declined	Payer / requester / requester	payment_request_received, _paid, _declined

4.5.7 Audit Trail

All audit entries are written by the internal function `write_audit`, which records the caller's identifier and role, an action name, the affected entity and a JSON metadata object. Clients cannot execute it; this was closed after the Phase 4 review found that customers could forge entries. The table has a read policy for administrators and no update or delete policy, so no client can alter the trail. It is append-only with respect to the application, but not cryptographically protected: a database superuser could still change it.

Table 17. Audited actions

Action	Written by
funds_transferred, bill_payment_completed	Money engine
deposit_completed, withdrawal_completed	Cash-in and cash-out functions
admin_user_status_changed, profile_role_changed	User status function; role-change trigger
admin_account_opened, admin_account_status_changed	Account functions
balance_adjustment, transaction_reversed	Adjustment and reversal functions
saved_recipient_added	Saved recipient function
scheduled_payment_created, _updated, _cancelled, admin_run_schedules_triggered	Schedule functions
biller_created, biller_status_changed	Biller functions
payment_request_created, _paid, _declined, _cancelled	Request functions

Some events are not audited: sign-in and sign-out, CSV exports, payment-request expiry, notification reads, exchange-rate updates and direct edits or deletions of saved recipients. Objective O5 is therefore met for money movements and administrative changes but not for every action in the system.

4.5.8 Payment Request Lifecycle

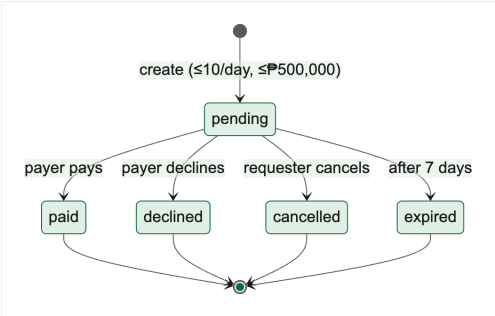


Figure 26. State diagram: payment request lifecycle

Figure 26 shows the five states of a payment request. Only a pending request can change, and paying it is idempotent: retrying with the same key after a timeout returns the original payment instead of an error, while any other attempt to pay a request that is no longer pending is refused. The linked `transaction_id` is unique, so one transaction can settle only one request.

4.6 Authentication, Security and Validation

4.6.1 Authentication

Customers and staff authenticate with email and password through Supabase Auth. Customers must confirm their email before the app can be used. Sessions are JSON Web Tokens that the clients refresh automatically: the mobile app stores them on the device, and the console stores them in cookies that the middleware refreshes on every request. The database identifies the caller from the token through `auth.uid()`.

4.6.2 Authorization

- **Row Level Security** is enabled on all 15 tables with 26 policies. Customers can read their own profile, accounts, ledger entries, transfers, notifications, saved recipients, schedules, bill payments and payment requests in which they are a party. Administrators can read all rows through policies that call `is_admin()`.
- **No client write access to money or audit tables.** There are no insert, update or delete policies on `transactions`, `ledger_entries`, `transfers` or `audit_logs`. Writes happen only inside SECURITY DEFINER functions with a fixed `search_path`.
- **Function permissions.** Phase 6 Stage B (in progress) revokes execute permission on every function from the public and anonymous roles by default and grants it to signed-in users only for the 29 functions the apps call. Internal functions such as `_transfer_core` and `write_audit` are executable by no client role.
- **Profile guard.** A trigger blocks anyone except an administrator or the trusted backend from changing a profile's role or status, so a customer cannot promote or unsuspend himself or herself.
- **Suspension.** `is_admin()` requires an active status, and every money function rejects a suspended caller, so suspension takes effect on the next request.

4.6.3 Validation Layers

Input is validated twice. The clients check formats and obvious errors to give fast feedback, but the database functions repeat every rule that matters for safety: ownership, status, limits, funds, reference formats and idempotency. A modified client therefore cannot bypass any rule. Table constraints provide a third layer, for example positive amounts, valid status values and unique account numbers.

4.6.4 Secrets and Configuration

Only the public anon key and project URL are shipped in the clients, through `EXPO_PUBLIC_` and `NEXT_PUBLIC_` environment variables. The service-role key is used only by the seed and test scripts and by the Edge Function.

Environment files are excluded from version control, and the continuous-integration workflow runs a gitleaks scan for leaked secrets. The exchange-rate secret is kept in Supabase Vault and in the Edge Function's secret settings.

4.6.5 Security Work in Progress

At the time of writing, Phase 6 Stage B is adding: default revocation of function permissions from anonymous users; removal of anonymous read access to billers and exchange rates; an automated audit that every table has RLS enabled; encrypted session storage on the phone; hiding the app's contents in the app switcher; stronger password rules; and a written security review. These are not yet reviewed and are described here as planned or in progress.

4.7 Integration of System Components

The three components interact only through Supabase. Figure 27 traces an immediate transfer from the customer's tap to the database and back. The app collects and validates input, generates the idempotency key, obtains confirmation, and calls the `transfer` function over HTTPS with the user's token. PostgREST runs the function, which performs the steps in Section 4.5.1 and returns the new transaction.

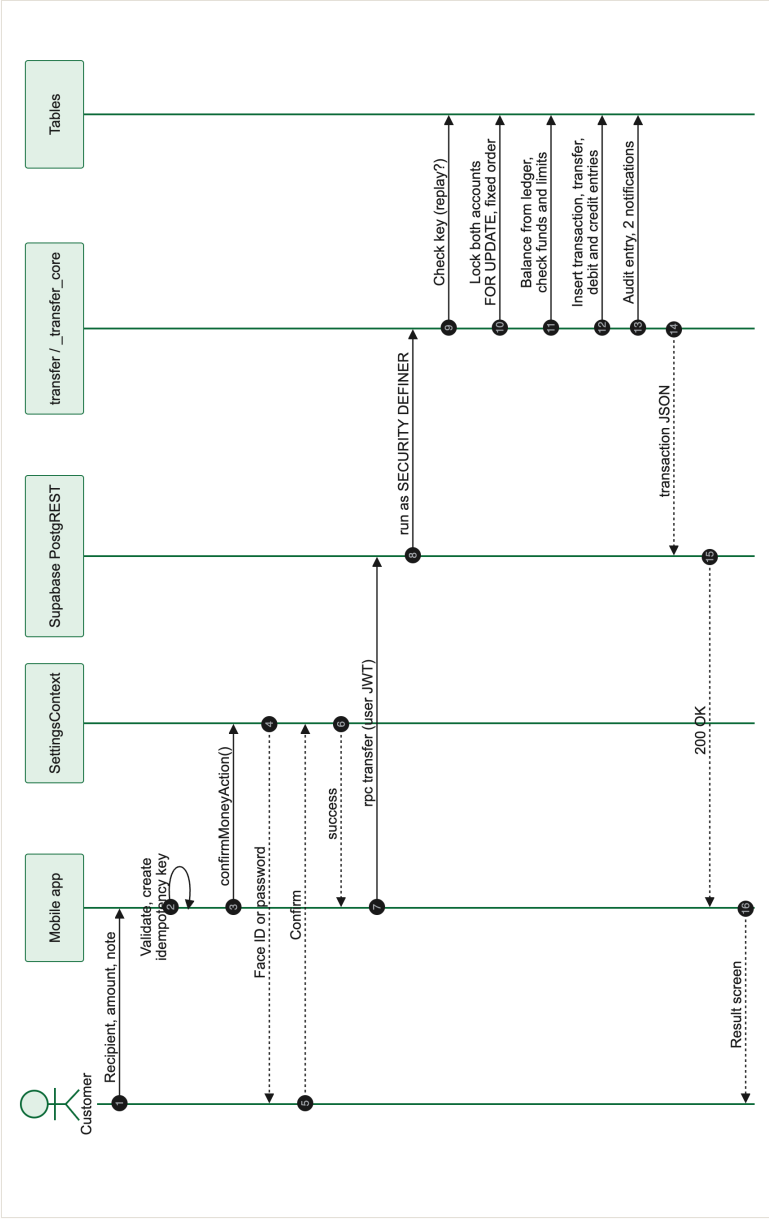


Figure 27. Sequence diagram: immediate transfer (rotated to fit the page)

Figure 28 traces a reversal from the staff console. The middleware confirms the administrator, the server action calls the reversal function with the administrator's token, and the function locks, validates, writes the compensating entries, audits and notifies. The affected customer sees the reversal in Activity and a notification in Inbox the next time the app loads, which shows how the console and the mobile app stay consistent through the shared database rather than through direct communication.

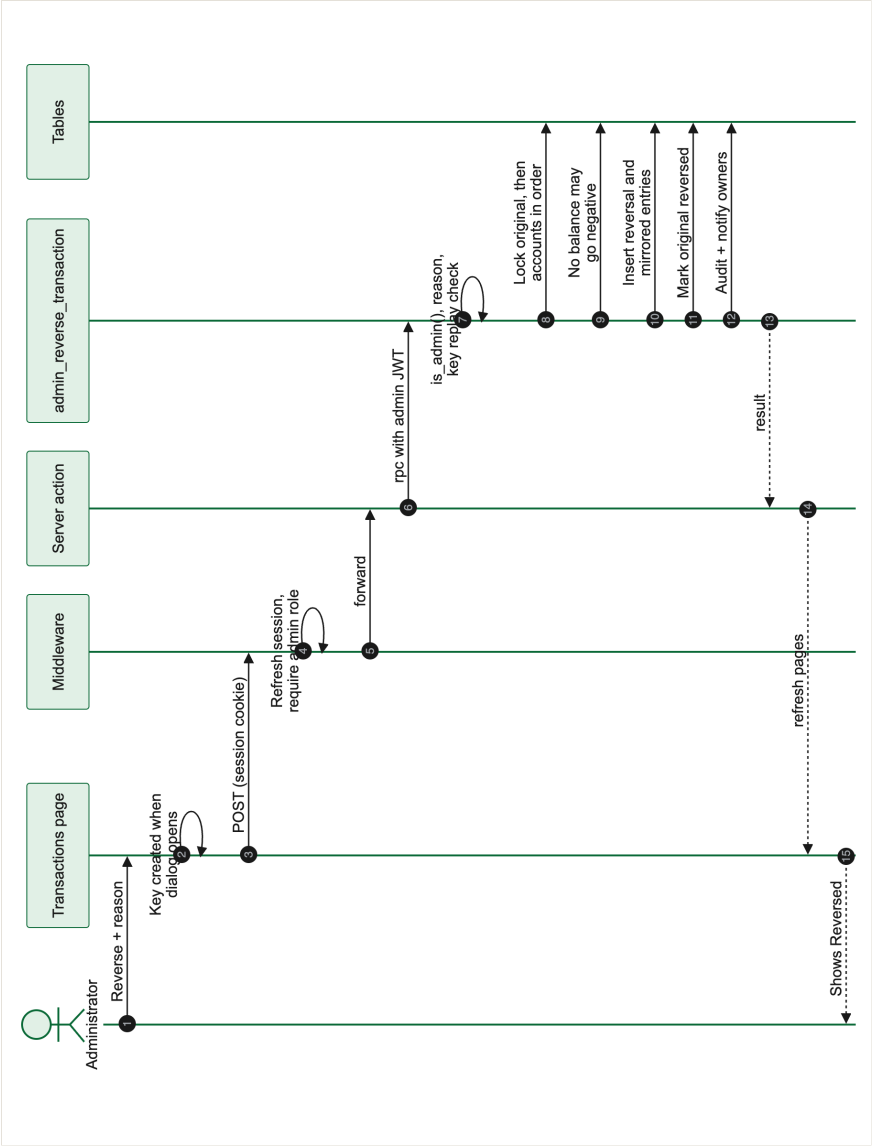


Figure 28. Sequence diagram: administrator reverses a transfer (rotated to fit the page)

Chapter 5. Testing and Evaluation

5.1 Testing Approach

Testing was organised in layers, from fast offline checks to observation of the running application. Because the implementing agent's early reports included results that had not touched a real database, this chapter distinguishes carefully between kinds of evidence. Every result is labelled with one of the codes in Table 18, and no result is reported as passing unless a recorded run supports it.

Table 18. Evidence types used in this chapter

Code	Evidence	Date and scope
H	Offline database harness (`npm run test:db`): applies all migrations to an in-process PostgreSQL (PGlite) with Supabase stand-ins, then runs each check as the relevant user	Run on 4 Nov 2026 against the working tree: 63 passed, 0 failed
L	Live suites (`npm run test:rls`, `test:money`, `test:admin`, `test:extras`, `test:schedules`, `test:qr`, `test:fx`) against the hosted Supabase project, observed by the reviewer	Run on 3 Nov 2026: 165 of 174 checks passed; all 9 failures were errors in the test scripts (Section 5.4)
U	Unit tests (`npm run test:unit`, Jest) for shared utilities, mobile components and admin components	Run on 4 Nov 2026: 50 of 50 tests in 7 suites passed
S	Observation of the running app on an iPhone 17 Pro Max simulator (Expo Go) and of the admin console in a browser	Mobile on 3 Nov 2026 (Phase 5 build); admin on 4 Nov 2026
C	Inspection of source code and migrations	4 Nov 2026
A	Reported by the implementing agent and not yet independently re-run	PHASE6_REPORT.md, 4 Nov 2026

The offline harness cannot test true concurrency because it has a single database connection, and it does not run pg_cron, pg_net or the Edge Function. Concurrency was therefore tested only against the live database. End-to-end flows exist for Maestro (mobile) and Playwright (admin) but no recorded run of them was available, so they are not counted as evidence.

5.2 Test Environment

- **Database:** hosted Supabase project with all migrations applied up to Phase 6 Stage 0; seeded with one administrator, two customers, the system profile and four demonstration billers.
- **Mobile:** Expo Go on an iPhone 17 Pro Max simulator (iOS 26.4, 440-point width), signed in as the seeded customer `customer1`.
- **Admin:** Next.js development server on a local machine, signed in as the seeded administrator, viewed in Chrome at 1,440 pixels wide.
- **Tooling:** Node.js 24.12.0; TypeScript type checking passed for all three workspaces and ESLint reported 0 errors and 95 warnings on 4 Nov 2026.

5.3 Test Cases

The tables below list representative test cases by category. The full list of 63 harness checks is in Appendix E. "Pass" means the observed result matched the expected result in the run cited.

5.3.1 Authentication and Authorization

Table 19. Test cases: authentication and authorization

ID	Objective and input	Expected result	Observed result	Ev.
TC-01	Sign up a new user	Profile and one savings account created automatically	Created for each test user. Pass	H
TC-02	Customer A reads Customer B's accounts	No rows returned	No rows. Pass	H, L

ID	Objective and input	Expected result	Observed result	Ev.
TC-03	Customer changes own role, then own status	Both rejected by the guard trigger	Rejected. Pass	H
TC-04	Anonymous caller changes a role; calls `transfer` and `deposit`	Rejected	Rejected with permission denied. Pass	H
TC-05	Customer inserts directly into `ledger_entries` or `audit_logs`	Rejected	Rejected. Pass	H, L
TC-06	Customer calls `_transfer_core` or `write_audit` directly	Permission denied	Denied. Pass	H, L
TC-07	Customer calls each admin function	"Admin privilege required"	Rejected. Pass	H, L
TC-08	Suspended customer transfers, deposits or withdraws	Rejected	Rejected. Pass	H, L
TC-09	Suspended administrator uses admin functions	Loses admin powers	Rejected. Pass	L
TC-10	Every public table has RLS enabled; no permissive "using (true)" policies except the health check	Audit passes	Passes (in-progress Stage B check). Pass	H
TC-11	Non-admin opens admin console routes	Redirected to /unauthorized	Playwright test written; no recorded run. Not run	—

5.3.2 Transaction Logic and Integrity

Table 20. Test cases: transaction logic and integrity

ID	Objective and input	Expected result	Observed result	Ev.
TC-12	Deposit, then repeat with the same idempotency key	One credit only; second call returns the original	One credit. Pass	H, L
TC-13	Transfer A to B, then repeat with the same key	Exact debit and credit once; replay returns original	Pass	H, L
TC-14	Transfer more than the available balance	"Insufficient funds"; no rows written	Rejected. Pass	H, L
TC-15	Spend from another customer's account	Rejected	Rejected. Pass	H
TC-16	Parallel withdrawals totalling more than the balance	Balance never below zero; rejections say insufficient funds	Balance stayed non-negative. Pass	L
TC-17	Two administrators reverse the same transfer at once	Exactly one reversal succeeds	One succeeded. Pass	L
TC-18	Reverse a transfer, then attempt a second reversal with a new key	Second attempt rejected	Rejected. Pass	H
TC-19	Run the scheduler twice for the same due date	Exactly one run and one debit	One run. Pass	H, L
TC-20	Scheduled run with insufficient funds	Run recorded as failed; no ledger rows	Pass	H
TC-21	Monthly schedule starting 31 January	Runs 28 Feb, 31 Mar, 30 Apr	Pass	H
TC-22	Pay a request, retry with same key, then pay again with a new key	Paid once; retry succeeds; new key rejected	Pass	H
TC-23	Reconciliation: each account balance against its ledger sum; each transfer, bill and reversal nets to zero	0 mismatches	0 mismatches (live: 9 accounts, 19 entries on 3 Nov; harness on 4 Nov). Pass	H, L

5.3.3 Input Validation and Business Rules

Table 21. Test cases: validation

ID	Objective and input	Expected result	Observed result	Ev.
TC-24	Pay a bill with a malformed reference	Rejected with the biller's format	Rejected. Pass	H, L

ID	Objective and input	Expected result	Observed result	Ev.
TC-25	Save own account or a non-existent account as a recipient	Both rejected	Pass	H, L
TC-26	Change a saved recipient's account number directly	Rejected; only nickname may change	Rejected. Pass	H
TC-27	Requester pays own payment request	Rejected	Pass	H, L
TC-28	Statement period over one year; non-owner fetches statement	Both rejected; owner's statement reconciles	Pass	H, L
TC-29	Parse QR codes with a foreign scheme, version 2, invalid account, invalid amount, or a note over 60 characters	Each rejected	Pass	U
TC-30	Map server errors to user messages	Plain-language messages for each known error	Pass	U
TC-31	Confirmation helper with biometrics disabled, unsupported or failed, and with a wrong password	Password required; never succeeds without a check	Pass	U

5.3.4 Administrative Functions

Table 22. Test cases: administrative functions

ID	Objective and input	Expected result	Observed result	Ev.
TC-32	Administrator suspends and reactivates a customer	Status changes; audit entry written	Pass	H, L
TC-33	Close an account with a non-zero balance	Rejected	Pass	L
TC-34	Credit adjustment with reason	Adjustment transaction, ledger credit and balance increase	Test script expected a field the function does not return and failed on 3 Nov; the adjustment itself succeeded. Test corrected in Phase 6	L, A
TC-35	Create a biller	Biller and settlement account created	Pass	H
TC-36	Open each admin page while signed in as administrator	Pages render with data	All eight pages rendered (Figures 17 to 22). Pass	S
TC-37	Deactivate a biller from the Billers page	Biller becomes inactive; audit entry written	Code inspection: update is silently ignored. Fail	C

5.3.5 User Interface Behaviour

The following cases were observed on the iOS simulator on 3 November 2026, before the Phase 6 fixes. Where the code has since changed, the change has not yet been re-verified on a device.

Table 23. Test cases: user interface (iOS simulator)

ID	Objective and input	Expected result	Observed result	Ev.
TC-38	Sign in as the seeded customer	Home screen with balances	Pass	S
TC-39	Cash-in ₱500	Balance increases; result screen shown	Balance increased by ₱500, but the sheet closed before the result could be read. Fail (fixed in code in Phase 6; not re-verified)	S, C
TC-40	Tap outside the sheet on the Transfer review step	Input preserved or confirmation asked	Transfer discarded with no warning. Fail (fixed in code in Phase 6; not re-verified)	S, C
TC-41	Open QR hub, Pay Bills, Scheduled and FX screens	Screens load with data	Pass; FX latest date was 26 Oct 2026 because the daily sync was not yet running	S
TC-42	Inbox badge on first launch	Unread count visible	Badge appeared only after Inbox was opened. Fail	S
TC-43	Hide balance, then open Activity	Amounts masked	Amounts remain visible. Fail	C

5.4 Actual Results

Table 24 summarises all recorded runs.

Table 24. Summary of recorded test runs

Run	Date	Result	Ev.
Offline database harness	4 Nov 2026	63 passed, 0 failed (56 committed checks plus 7 in-progress Stage B checks)	H
Unit tests	4 Nov 2026	50 passed in 7 suites	U
Type checking (3 workspaces)	4 Nov 2026	Passed	C
Lint	4 Nov 2026	0 errors, 95 warnings (mostly use of `any`)	C
Live suites, first real run	3 Nov 2026	165 of 174 checks passed	L
Live concurrency checks	3 Nov 2026	Parallel overdraft, double reversal and scheduler run-once all passed	L
Live suites after test fixes	4 Nov 2026	179 of 179 reported	A
Maestro and Playwright flows	—	Written; no recorded run	—

The nine failures in the first live run were investigated individually. Each was an error in the test script, not in the system: the payment-request test used wrong parameter names; the exchange-rate test checked source text for an outdated column type; the adjustment test expected a `new_balance` field the function does not return; the audit-delete test expected an error, whereas Row Level Security silently matches no rows (which is the correct protection); the withdrawal test used an amount above the ₱5,000,000 limit, so it hit the limit rule instead of the funds rule; and the schedule test asserted that system-wide credits equal debits, which is not true in this ledger model (Section 4.5.2). The scripts were corrected in Phase 6 Stage 0; the corrected run of 179 of 179 is reported by the implementing agent and is awaiting independent re-run.

5.5 System Evaluation

Table 25 evaluates the system against the success metrics defined in the proposal.

Table 25. Evaluation against the proposal's success metrics

Metric	Target	Result	Assessment
Balance integrity	0 inconsistencies in reconciliation	0 mismatches live (3 Nov) and in the harness (4 Nov)	Met
Transfer atomicity	100% all-or-nothing	Single-transaction functions; failed runs write no ledger rows	Met
Idempotency	Duplicate submission gives exactly one transaction	Replays return the original for every money function	Met in tests; one client-side path can issue a new key after success (Section 5.6)
Access control	100% of unauthorized attempts blocked	All cross-user, anonymous and privilege tests blocked	Met in tests
Audit coverage	100% of money movements and admin actions	All money movements and admin changes audited; some secondary events are not	Largely met
Feature completeness	100% of in-scope features end to end	21 of 26 functional requirements implemented; 3 partial; 2 not implemented (Table 5)	Partially met
Performance	Key screens under 2 s	Not measured	Not yet evaluated
Phase discipline	Every phase reviewed before the next	All phases reviewed; three fix passes issued	Met

5.6 Known Issues and Limitations

Table 26 lists open defects known on 4 November 2026. Severity reflects the effect on a user or on data integrity in the prototype.

Table 26. Known issues

ID	Area	Issue	Severity
KI-01	Mobile, Transfer	After a QR-prefilled transfer succeeds, the refreshed Home data re-runs the sheet's reset logic, which returns to the review step with a new idempotency key; tapping again would send a second transfer	High
KI-02	Mobile, Transfer	The result screen is replaced by the first step about a second after success; the receipt shows the pre-transfer balance	Medium
KI-03	Mobile, privacy	Hide balance is not applied in Activity	Medium
KI-04	Mobile, design	Brand font not applied; several text colors below WCAG AA contrast; touch targets under 44 points	Medium
KI-05	Mobile, auth	Password reset cannot be completed inside the app	Medium
KI-06	Mobile, QR	Entering a QR amount above ₱500,000 causes a rendering error	Medium
KI-07	Admin, billers	Activate/Deactivate silently changes nothing and is not audited; the in-progress Stage B migration also hides inactive billers from administrators	Medium
KI-08	Admin, dashboard	"Failed runs" always shows zero; "30-day volume" shows all-time volume	Medium
KI-09	Backend, FX sync	It has not been confirmed whether the Edge Function was deployed with JWT verification disabled, which the scheduled call requires	Medium (unverified)
KI-10	Mobile	Inbox badge empty until Inbox is opened; data not refreshed on tab focus	Low
KI-11	Mobile, Android	Renaming a saved recipient uses an iOS-only dialog	Low
KI-12	Mobile, requests	Authorization prompt shows an internal account identifier; source account chosen automatically	Low
KI-13	Mobile, wording	Raw identifiers and enum values appear in some rows; database-written notifications use "PHP 500.00" and administrator wording	Low
KI-14	Admin	Close-account dialog labelled "Unfreeze"; a failed manual schedule run is shown in success style; audit view limited to 100 entries; tables unpaginated beyond 1,000 rows	Low
KI-15	Admin	Middleware and export check the role but not the status (the database still refuses suspended administrators)	Low
KI-16	Backend	Adjustment notifications truncate centavos; the daily payment-request limit resets at 08:00 Manila time rather than midnight; failed scheduled runs count toward the maximum	Low
KI-17	Design limitation	Cash-in, cash-out and adjustments are single-sided ledger entries (Section 4.5.2)	Design

Several defects observed on 3 November were addressed in code by Phase 6 Stage 0 and are not repeated above: result screens closing early, unguarded dismissal of sheets during a transfer, missing idempotency key on QR payments, failure of scheduled transfers to other customers, the admin "Add New Biller" error, unmapped server errors, and the balance card showing the total under one account's name. They still require confirmation on a device.

5.7 Outstanding Verification

- Independent re-run of the corrected live suites and of the Maestro and Playwright flows.
- Device testing on Android and on physical phones, including QR scanning with a real camera.
- Accessibility testing with VoiceOver and TalkBack and at large text sizes.
- Measurement of screen load times against the two-second target.
- A dropped-connection test proving that a retried transfer creates exactly one transaction (planned in Phase 6 Stage C).
- Confirmation that the scheduled exchange-rate sync and the five-minute scheduler run successfully on the hosted project.
- Review of the Phase 6 Stage B security changes once committed.

Chapter 6. Summary, Conclusions and Recommendations

6.1 Summary of Accomplishments

In about two weeks of phased development, from 20 October to 4 November 2026, the team delivered a working banking prototype with a customer mobile app, a staff console and a shared database. The core of the system, its money engine, ledger and access rules, was designed first, reviewed at every phase gate, and tested both offline and against the live database. On top of that core, the system supports the full set of everyday functions planned in the proposal and the eight features added by Scope Change 01, presented in an interface built on Davao City's Kadayawan identity.

6.2 Alignment With the Original Objectives

Table 27. Achievement of the specific objectives

ID	Objective	Status	Evidence
O1	Secure authentication and session management	Achieved	Supabase Auth; email verification gate; no unauthenticated data access (TC-02, TC-04)
O2	Role-based access	Achieved in tests	RLS on all tables; admin checks in functions and middleware (TC-02 to TC-10)
O3	Atomic, correct money movement	Achieved in tests	Single-transaction functions, row locks, zero reconciliation mismatches (TC-12 to TC-23)
O4	Prevent duplicate transactions	Achieved on the server; one client defect open	Idempotency keys and unique constraints (TC-12, TC-13, TC-22); KI-01
O5	Complete, tamper-evident audit trail	Largely achieved	All money and admin changes audited; client cannot write; some events not audited; not cryptographically sealed
O6	Reliable, responsive experience	Partially achieved	Errors mapped and flows guarded; load time not measured; UI defects open
O7	Complete, demonstrable prototype	Largely achieved	21 of 26 functional requirements implemented; deployment pending

6.3 Conclusions

The evidence supports four conclusions. First, placing all money logic in database functions that lock accounts and run in one transaction prevented the integrity failures the project set out to avoid: concurrent withdrawals could not overdraw an account, a transfer could not be reversed twice, the scheduler could not pay the same instruction twice, and reconciliation found no mismatches.

Second, enforcing access in the database rather than in the user interface made the rules hold for every client. Customers could not read other customers' data, write to money or audit tables, or call internal or administrative functions, even when calling the API directly.

Third, idempotency keys checked by the server make retries safe, but the client must also handle keys correctly. The open defect KI-01 shows that the server's guarantee can be undermined if a client generates a fresh key for the same intended payment.

Fourth, the review process was essential. Several serious issues, including a forgeable audit function, a double-reversal race, unenforced suspension and test reports that had not touched a database, were found only because each phase was checked against the real code and database rather than the implementer's summary.

6.4 Current Limitations

Davao Bank remains a simulation. It moves no real money, connects to no payment network, performs no identity verification and has not been deployed beyond development tooling. Its ledger is single-sided for cash-in, cash-out

and adjustments. Testing has concentrated on the database and on an iOS simulator; Android, physical devices, accessibility tools and performance have not yet been evaluated. The known issues in Section 5.6 remain open.

6.5 Recommendations

6.5.1 Immediate

1. Fix KI-01 and KI-02 by keeping the result screen until the user taps Done and by resetting the transfer sheet only when it opens, never in response to data refreshes.
2. Route the biller Activate/Deactivate button through `admin_set_biller_active`, and restore administrator visibility of inactive billers in the Stage B migration.
3. Correct the dashboard figures and apply "hide balance" throughout the app.
4. Complete Phase 6 Stages B and C and independently re-run every live suite and end-to-end flow.

6.5.2 Next Phases

1. Phase 7 (approved): refine the interface, apply the brand typeface and accessible contrast, unify flows, add a consistent motion system, and introduce the Kada and Duri illustrations for empty and error states.
2. Add a system "cash" account so that cash-in, cash-out and adjustments become balanced double entries, which would allow a system-wide zero-sum check.
3. Add pagination to admin tables and to the audit log, an audit-log export, and the read-only auditor role described in the proposal.
4. Deploy the admin console and build installable mobile apps, then measure performance against the two-second target.

6.5.3 Toward Production (Outside the Capstone Scope)

A production bank would additionally require a banking or e-money licence, identity verification and anti-money-laundering screening, integration with regulated payment rails, fraud monitoring, high-availability infrastructure, and an independently audited core ledger. These are recorded so that the prototype is not mistaken for a production system.

Appendix A. Data Dictionary

This appendix describes every table in the `public` schema in its final state after all migrations. Money amounts are `bigint` centavos. Unless stated, foreign keys take no action on delete. "NN" means NOT NULL.

profiles

One row per authenticated user, created by the sign-up trigger.

Table 28. Data dictionary: profiles

Column	Type	Constraints and notes
id	uuid	PK; FK to auth.users.id, cascade on delete
full_name	text	Nullable
phone	text	Nullable
role	text	NN, default customer; one of customer, admin, system
status	text	NN, default active; one of active, suspended
created_at	timestamptz	NN, default now()

RLS: users read and update their own row (role must remain customer); administrators read and update all rows. Triggers: guard against role or status changes by non-administrators; audit of role changes.

accounts

Customer deposit accounts and biller settlement accounts. There is no balance column.

Table 29. Data dictionary: accounts

Column	Type	Constraints and notes
id	uuid	PK, default gen_random_uuid()
account_number	text	NN, unique; "10" followed by 8 random digits
user_id	uuid	NN; FK to profiles.id, cascade
type	text	NN, default savings; savings or checking
currency	text	NN, default PHP
status	text	NN, default active; active, frozen or closed
created_at	timestamptz	NN, default now()

Indexes: user_id; account_number. RLS: owners and administrators may read.

transactions

The envelope of every money event.

Table 30. Data dictionary: transactions

Column	Type	Constraints and notes
id	uuid	PK
type	text	NN; transfer, deposit, withdrawal, adjustment, reversal, bill_payment
status	text	NN, default completed; pending, completed, failed, reversed
initiated_by	uuid	FK to profiles.id; nullable
idempotency_key	text	Unique
reference	text	Human-readable description
created_at	timestamptz	NN, default now()
reverses_transaction_id	uuid	FK to transactions.id; partial unique index ensures one reversal per transaction

RLS: a customer may read a transaction that has a ledger entry on one of his or her accounts; administrators read all.

ledger_entries

Debit and credit lines; the source of truth for balances.

Table 31. Data dictionary: ledger_entries

Column	Type	Constraints and notes
id	uuid	PK
account_id	uuid	NN; FK to accounts.id
transaction_id	uuid	NN; FK to transactions.id, cascade
direction	text	NN; debit or credit
amount	bigint	NN; check amount > 0 (centavos)
created_at	timestamptz	NN, default now()

Indexes: account_id; transaction_id. RLS: entries on own accounts; administrators read all. No client insert, update or delete.

transfers

Source and destination of transfer-type transactions, bill payments, request payments and their reversals.

Table 32. Data dictionary: transfers

Column	Type	Constraints and notes
id	uuid	PK
transaction_id	uuid	NN; FK to transactions.id, cascade
from_account	uuid	NN; FK to accounts.id
to_account	uuid	NN; FK to accounts.id; check from_account <> to_account
amount	bigint	NN; check amount > 0

audit_logs

Append-only record of sensitive actions, written only through write_audit.

Table 33. Data dictionary: audit_logs

Column	Type	Constraints and notes
id	uuid	PK
actor_id	uuid	Caller identifier (no foreign key; null for scheduled jobs)
actor_role	text	Caller role, or system
action	text	NN; for example funds_transferred
entity, entity_id	text	Affected table and row
metadata	jsonb	Details such as amounts, reasons and balances before and after
created_at	timestamptz	NN, default now()

RLS: administrators read; no client writes.

notifications

In-app messages.

Table 34. Data dictionary: notifications

Column	Type	Constraints and notes
id	uuid	PK
user_id	uuid	NN; FK to profiles.id, cascade
type	text	Event type (Table 16)
message	text	NN
read	boolean	NN, default false

Column	Type	Constraints and notes
created_at	timestamptz	NN, default now()

RLS: users read and update their own; administrators read all.

saved_recipients

A customer's saved destination accounts.

Table 35. Data dictionary: saved_recipients

Column	Type	Constraints and notes
id	uuid	PK
owner_id	uuid	NN; FK to profiles.id, cascade
account_number	text	NN; validated when added (no foreign key); unique per owner
nickname	text	NN; 1 to 40 characters
created_at	timestamptz	NN

RLS: owners read, delete and update; a column grant limits updates to nickname. Inserts only through add_saved_recipient.

billers

Demonstration billers.

Table 36. Data dictionary: billers

Column	Type	Constraints and notes
id	uuid	PK
name	text	NN, unique
category	text	NN; electricity, water, internet, mobile, other
account_id	uuid	NN, unique; FK to accounts.id (settlement account)
reference_label	text	NN; label shown to the customer
reference_pattern	text	NN; regular expression the reference must match
is_active	boolean	NN, default true
created_at	timestamptz	NN

RLS: signed-in users read active billers.

bill_payments

Biller and reference for each bill-payment transaction.

Table 37. Data dictionary: bill_payments

Column	Type	Constraints and notes
id	uuid	PK
transaction_id	uuid	NN, unique; FK to transactions.id, cascade
biller_id	uuid	NN; FK to billers.id
payer_account	uuid	NN; FK to accounts.id
biller_reference	text	NN
created_at	timestamptz	NN

scheduled_payments

One-time and recurring instructions.

Table 38. Data dictionary: scheduled_payments

Column	Type	Constraints and notes
id	uuid	PK
owner_id	uuid	NN; FK to profiles.id, cascade
from_account	uuid	NN; FK to accounts.id
kind	text	NN; transfer or bill (with matching target columns)
to_account / biller_id / biller_reference	uuid / uuid / text	Target, depending on kind
amount	bigint	NN; > 0
note	text	Optional
frequency	text	NN; once, weekly, monthly
start_date, end_date	date	end_date null or not before start_date
max_runs, runs_done	int	max_runs > 0 if set; runs_done default 0
next_run_date	date	NN
status	text	NN; active, completed, cancelled
created_at, updated_at	timestamptz	NN

scheduled_payment_runs

One execution record per schedule per date.

Table 39. Data dictionary: scheduled_payment_runs

Column	Type	Constraints and notes
id	uuid	PK
schedule_id	uuid	NN; FK to scheduled_payments.id, cascade; unique with run_date
run_date	date	NN
status	text	NN; completed or failed
transaction_id	uuid	FK to transactions.id; null when failed
failure_reason	text	Error message when failed
created_at	timestamptz	NN

payment_requests

Money requests between customers.

Table 40. Data dictionary: payment_requests

Column	Type	Constraints and notes
id	uuid	PK
requester_id	uuid	NN; FK to profiles.id, cascade
requester_account	uuid	NN; FK to accounts.id (account to credit)
payer_account_number	text	NN
payer_user_id	uuid	NN; FK to profiles.id
amount	bigint	NN; > 0
note	text	Up to 60 characters
status	text	NN, default pending; pending, paid, declined, cancelled, expired
transaction_id	uuid	Unique; FK to transactions.id
expires_at	timestamptz	NN, default now() + 7 days
created_at, responded_at	timestamptz	Creation and response times

RLS: requester, payer and administrators read. No client writes.

fx_rates

Daily ECB reference rates for display.

Table 41. Data dictionary: fx_rates

Column	Type	Constraints and notes
base, quote, rate_date	text, text, date	Composite PK
rate	numeric(18,8)	NN; > 0
source	text	NN, default "Frankfurter / European Central Bank"
fetched_at	timestamptz	NN

RLS: signed-in users read. Writes only through upsert_fx_rates by the service role.

health_check

Connectivity probe.

Table 42. Data dictionary: health_check

Column	Type	Constraints and notes
id	integer	PK, identity
status	text	NN; seeded "ok"
checked_at	timestamptz	NN

Views: `account_balances` (one row per account with balance, ledger entry count and last activity) and `account_transactions` (one row per ledger line with signed amount). Both run with the caller's permissions, so Row Level Security applies.

Appendix B. Database Function Reference

Table 43. Customer-callable functions

Function	Purpose
is_admin()	True if the caller is an active administrator
ensure_my_profile()	Create the caller's profile and account if missing
transfer(...), transfer_by_account_number(...)	Immediate transfer by account identifier or number
deposit(...), withdraw(...)	Simulated cash-in and cash-out
get_account_transactions(...)	History with signed amounts and counterparties
get_account_statement(...)	Opening balance, period transactions, closing balance
add_saved_recipient(...)	Validated recipient save
pay_bill(...)	Bill payment through the engine
create_scheduled_payment(...), create_scheduled_transfer_to_number(...)	Create a schedule
list_my_scheduled_payments(), update_scheduled_payment(...), cancel_scheduled_payment(...)	Manage schedules
create_payment_request(...), pay_payment_request(...), decline_payment_request(...), cancel_payment_request(...)	Payment request lifecycle

Table 44. Administrator functions

Function	Purpose
admin_get_users()	All users including logins without a profile
admin_get_dashboard_stats()	Dashboard figures
admin_set_user_status(...)	Suspend or reactivate with reason
admin_open_account(...)	Open an account for a user
admin_set_account_status(...)	Freeze, unfreeze or close with reason
admin_adjust_balance(...)	Audited credit or debit
admin_reverse_transaction(...)	Compensating reversal
admin_create_biller(...), admin_set_biller_active(...)	Biller management
admin_run_due_schedules_now()	Run the scheduler immediately

Table 45. Internal and trigger functions (not callable by clients)

Function	Purpose
_transfer_core(...)	The single money engine
write_audit(...)	Insert an audit entry
run_due_scheduled_payments()	Scheduled executor
expire_due_payment_requests()	Expire requests past seven days
upsert_fx_rates(...)	Store exchange rates (service role)
register_cron_jobs()	(Re)register the three scheduled jobs
handle_new_user()	Sign-up trigger: profile and account
guard_profile_self_update()	Block role and status changes by non-administrators
audit_profile_changes()	Audit role changes
audit_account_changes()	Legacy; its trigger was removed in Phase 4

Appendix C. Row Level Security Policies

Table 46. Row Level Security policies by table

Table	Policies
profiles	Read own; update own (role stays customer); admins read all; admins update all
accounts	Read own; admins read all
transactions	Read those touching own accounts; admins read all
ledger_entries	Read entries on own accounts; admins read all
transfers	Read those involving own accounts; admins read all
audit_logs	Admins read
notifications	Read own; update own; admins read all
saved_recipients	Read own; delete own; update own (nickname only by column grant)
billers	Signed-in users read active billers
bill_payments	Payer or admin reads
scheduled_payments	Owner or admin reads
scheduled_payment_runs	Owner of the schedule or admin reads
payment_requests	Requester, payer or admin reads
fx_rates	Signed-in users read
health_check	Anyone reads

Appendix D. Diagram Sources

All diagrams in this document were produced from Mermaid text definitions that describe the implemented system. The definitions are supplied with this document in the folder `diagram-sources/`, one `.mmd` file per diagram (the architecture diagram is a hand-drawn `.svg`), so they can be edited and regenerated with any Mermaid-compatible tool. Table 47 maps each file to its figure.

Table 47. Diagram source files

File	Figure
architecture.svg	Figure 1
use-case-customer.mmd	Figure 2
use-case-staff.mmd	Figure 3
context-dfd.mmd	Figure 4
dfd-level1.mmd	Figure 5
erd-core.mmd	Figure 6
erd-features.mmd	Figure 7
mobile-navigation.mmd	Figure 8
admin-workflow.mmd	Figure 15
activity-transfer.mmd	Figure 23
state-transaction.mmd	Figure 24
activity-scheduler.mmd	Figure 25
state-payment-request.mmd	Figure 26
sequence-transfer.mmd	Figure 27
sequence-reversal.mmd	Figure 28

Appendix E. Offline Database Test Inventory

The 63 checks run by `npm run test:db` on 4 November 2026, all of which passed. Checks 55 to 61 belong to the in-progress Phase 6 Stage B work.

Table 48. Offline harness checks

No.	Check
1	Signup trigger provisions profile and account for each user
2	Service role can promote a profile to admin (seed path)
3	Customer changing own role is rejected
4	Customer changing own status is rejected
5	Anonymous role change is rejected
6	Admin can suspend a customer
7	Database superuser can update profile status
8	Customer deposit works
9	Deposit idempotent replay (same key, no second credit)
10	Transfer A to B
11	Transfer idempotent replay
12	Insufficient funds rejected
13	Cannot spend from someone else's account
14	Customer cannot call _transfer_core
15	Customer cannot forge audit log
16	Customer cannot write ledger directly
17	RLS: B cannot see A's accounts
18	Suspended customer cannot transfer
19	Suspended customer cannot un-suspend self
20	Admin reversal works
21	Second reversal (new key) rejected
22	Customer cannot call admin functions
23	Cannot save own account as recipient
24	Cannot save nonexistent account
25	Save valid recipient
26	add_saved_recipient does not return holder name
27	Direct insert into saved_recipients denied
28	Owner cannot change a saved recipient's account number
29	Statement returns opening, closing and rows
30	Non-owner cannot fetch statement
31	Admin creates biller and settlement account
32	Bill pay rejects malformed reference
33	Bill pay with valid reference
34	bill_payments row linked to its transaction
35	Create weekly schedule starting today
36	Executor called twice gives exactly one run and one debit
37	next_run_date advanced by 7 days
38	Unfunded scheduled run recorded as failed, no ledger writes
39	Monthly from 31 Jan gives 28 Feb, 31 Mar, 30 Apr
40	Customer cannot run the executor
41	Customer schedules to another customer by account number

No.	Check
42	Scheduling to a nonexistent number returns not-found
43	Scheduling from someone else's account is rejected
44	list_my_scheduled_payments returns only caller rows with last four digits
45	Create payment request
46	Request response has no names
47	Requester cannot pay own request
48	Payer pays request (requester credited)
49	Request marked paid and linked to transaction
50	Paid request same-key retry returns success
51	Second payment (new key) rejected, no extra money moved
52	Direct insert into payment_requests denied
53	Customers can read fx_rates (768 rows)
54	Customers cannot write fx_rates
55	Anonymous transfer call rejected
56	Anonymous deposit call rejected
57	Anonymous users cannot read billers
58	Anonymous users cannot read fx_rates
59	RLS audit: all public tables enable row-level security
60	RLS audit: no "using (true)" policies outside health_check
61	RLS audit: neither billers nor fx_rates allow anonymous reads
62	Reconciliation: every transfer, reversal and bill nets to zero
63	Reconciliation: account balance equals ledger sum for every account

Appendix F. Installation and Configuration Guide

These steps set up a development copy. Secret values are never written in this document; they are obtained from the Supabase dashboard of the project.

1. Install Node.js 20 or later (24.12.0 was used) and run `npm install` in the repository root.
2. Copy `.env.example` to `.env` (root), `apps/admin/.env.local` and `apps/mobile/.env`, and fill in `SUPABASE_URL`, `SUPABASE_ANON_KEY`, `EXPO_PUBLIC_SUPABASE_URL`, `EXPO_PUBLIC_SUPABASE_ANON_KEY`, `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY` and, for scripts only, `SUPABASE_SERVICE_ROLE_KEY`. Optionally set `SEED_PASSWORD`.
3. Link the hosted project and apply migrations: `npx supabase login`, `npx supabase link --project-ref <project-ref>`, then `npx supabase db push --remote`. For a local database use `npx supabase db reset`.
4. Seed demonstration data: `npm run seed`. This creates an administrator, two customers, the system profile and four demonstration billers.
5. Enable the `pg_cron` and `pg_net` extensions in the Supabase dashboard, deploy the Edge Function with `npx supabase functions deploy fx-sync`, store the sync secret with `npx supabase secrets set FX_SYNC_SECRET=<value>` and in Vault under the same name, and run `select public.register_cron_jobs();` in the SQL editor.
6. Run the admin console with `npm run dev:admin` (http://localhost:3000) and the mobile app with `npm run dev:mobile`, then open it in Expo Go.
7. Verify with `npm run typecheck`, `npm run lint`, `npm run test:unit` and `npm run test:db`; the live suites (`npm run test:rls` and others) require the hosted database.

```
# Common commands
npm install
npm run seed
npm run dev:admin
npm run dev:mobile
npm run test:db
npm run test:unit
```

Appendix G. User Guide (Customer)

Getting started

1. Open the app in Expo Go and tap **Sign Up**. Enter your name, email and a password, then confirm your email from the link you receive.
2. Return to the app and **Sign In**. Your savings account is created automatically.
3. Use **Cash-in** to add play money for testing. All money in Davao Bank is simulated.

Sending money

1. On Home, tap **Transfer**. Choose a saved recipient, one of your accounts, or enter a 10-digit account number.
2. Enter the amount and an optional note. Choose **Now**, **Later** or **Recurring**.
3. Check the review screen, tap the confirm button, and approve with Face ID, fingerprint or your password.

Other tasks

- **Pay a bill:** Home, **Pay Bills**, choose a demo biller, enter the reference from your bill and the amount.
- **Receive money:** Home, **QR**, **My QR**, and let the payer scan it. To ask for money, use the **Request** tab.
- **Pay a request:** tap the pending-request banner on Home, then **Pay Now**.
- **See history and receipts: Activity** tab; tap any row. Tap **Statement** to export a PDF or CSV.
- **Hide your balance:** tap the eye icon on the balance card.
- **Manage scheduled payments, saved recipients and biometrics: Profile** tab.

Appendix H. Administrator Guide

1. Open the console and sign in with an administrator account. Non-administrators are refused.
2. **Dashboard:** review system figures and the latest transactions and audit entries.
3. **Users:** search for a customer; use **Suspend** or **Activate** (a reason is required); use **+ Account** to open another account.
4. **Accounts:** use **Freeze**, **Unfreeze** or **Close** (reason required; closing needs a zero balance). Use **Adjust Balance** to credit or debit with a reason.
5. **Transactions:** inspect any transaction; use **Reverse** on a completed transfer (reason required); use **Export Ledger CSV** for a download.
6. **Schedules:** view run history, **Run Due Schedules Now**, or cancel a schedule with a reason.
7. **Billers:** add a demo biller with a reference label and pattern.
8. **Audit Log:** filter recent actions and open any entry to see its details.

Every action that changes data is recorded in the audit log under the signed-in administrator's identity.

Appendix I. References

- Ancheta, R. A., & Arcete, J. (2026). *Davao Bank: A mobile banking system rooted in Davao City culture* (Project proposal, version 1.1).
- Davao Bank project team. (2026). *Scope Change 01: Feature additions*; phase implementation prompts (Phases 1 to 7); *PHASE5_REPORT.md* and *PHASE6_REPORT.md*. Project repository.
- European Central Bank. *Euro foreign exchange reference rates*. https://www.ecb.europa.eu/stats/policy_and_exchange_rates/euro_reference_exchange_rates/html/index.en.html
- Expo. *Expo documentation*. https://docs.expo.dev/
- Frankfurter. *Frankfurter exchange rates API*. https://frankfurter.dev/
- Meta Platforms. *React Native documentation*. https://reactnative.dev/
- Republic of the Philippines. (2000). *Republic Act No. 8791, The General Banking Law of 2000*.
- PostgreSQL Global Development Group. *Explicit locking*. https://www.postgresql.org/docs/current/explicit-locking.html
- PostgreSQL Global Development Group. *CREATE FUNCTION*. https://www.postgresql.org/docs/current/sql-createfunction.html
- React Navigation. *React Navigation documentation*. https://reactnavigation.org/
- Supabase. *Row Level Security*. https://supabase.com/docs/guides/database/postgres/row-level-security
- Supabase. *Database functions*. https://supabase.com/docs/guides/database/functions
- Supabase. *pg_cron extension*. https://supabase.com/docs/guides/database/extensions/pg_cron
- Supabase. *Edge Functions*. https://supabase.com/docs/guides/functions
- Vercel. *Next.js documentation*. https://nextjs.org/docs
- World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/